Mini project report on

Digital Rights Management and Asset Licensing Ledger

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Shreyas B Patil	**PES2UG24CS488**
Shreyas Patil	**PES2UG24CS492**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Digital Rights Management and Asset Licensing Ledger

is a bonafide work carried out by

Shreyas B Patil	**PES2UG24CS488**
Shreyas Patil	**PES2UG24CS492**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CSS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Digital Rights Management and Asset Licensing Ledger** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Shreyas B Patil	**PES2UG24CS488**
Shreyas Patil	**PES2UG24CS492**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

The **Digital Rights Management and Asset Licensing Ledger** is a database management system developed to manage digital assets, licensing agreements, ownership details, payments, renewals, and usage rights. As digital content such as movies, music, software, and books is distributed across different platforms and regions, managing licensing information becomes increasingly important. Our project aims to provide a centralized system that makes it easier to organize, monitor, and manage these licensing activities.

The project uses **MySQL** as the primary relational database to store information about digital assets, owners, licensees, territories, rights, licensing agreements, payments, and renewals. Relationships between these entities are maintained using primary keys, foreign keys, and database constraints. We have also implemented advanced SQL features such as stored procedures, functions, triggers, views, and transactions to support licensing operations and maintain data consistency. SQL queries are used to retrieve information such as active licenses, revenue generated, upcoming license expirations, and licensing conflicts.

Along with MySQL, the system uses **MongoDB** to maintain digital asset access logs and licensing violation reports. Whenever an access attempt is simulated, the system checks the licensing information and records whether access is allowed or denied. If a violation is detected, such as an attempt to access an asset from an unauthorized territory, a violation report is generated. The system also allows authorized users to review and resolve these violations while preserving the original records.

The application is developed using **Next.js**, which provides the frontend interface and backend API functionality. The interface allows users to view digital assets, manage licensing agreements, monitor payments and renewals, and perform other database operations. Authentication and role-based access control are implemented to separate the responsibilities of the System Administrator and License Manager. Database-level roles and permissions are also used to restrict access to specific operations.

Overall, the project demonstrates how relational and NoSQL databases can be integrated into a single application to manage digital licensing information. It combines important DBMS concepts with a working web interface, providing a practical approach to organizing licensing records, tracking asset usage, and maintaining accountability in digital rights management.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	12
2.	PROBLEM DEFINITION	14
3.	ER MODEL	17
4.	ER TO RELATIONAL MAPPING	20
5.	DDL STATEMENTS	26
6.	DML STATEMENTS	33
7.	QUERIES	41
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	47
9.	FRONT END DEVELOPMENT	53
10.	MONGODB IMPLEMENTATION	60
11.	AUTHENTICATION AND ROLE BASED ACCESS CONTROLS	67
12.	CONCLUSION	69
REFERENCES/BIBLIOGRAPHY		70

LIST OF TABLES

Table No.	Title	Page No.
2.1	Functional requirements of the mini project	15-16
3.1	Summary of entities and attributes	18-19
3.2	Relationships and cardinality ratios	19

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER Diagram of Digital Rights Management and Asset Licensing Ledger	18
4.1	Relational Schema Diagram of Digital Rights Management and Asset Licensing Ledger	22
5.1	DDL statement for drm_licensing database	26
5.2	DDL statement for owner table	27
5.3	DDL statement for licensee table	27
5.4	DDL statement for territory table	28
5.5	DDL statement for right_ent table	28
5.6	DDL statement for asset table	29
5.7	DDL statement for license table	29
5.8	DDL statement for license_right table	30
5.9	DDL statement for renewal table	30
5.10	DDL statement for payment table	31
5.11	DDL statement for license_history table	31
5.12	DDL Statement for the app_user Table	32
6.1	DML statement for owner table	33
6.2	DML statement for licensee table	33

6.3	DML statement for territory table	34
6.4	Insert DML statement for right_ent table	34
6.5	Select DML statement for right_ent table	35
6.6	Insert DML statement for asset table	35
6.7	Select DML statement for asset table	36
6.8	Insert DML statement for license table	36
6.9	Select DML statement for license table	37
6.10	Insert DML statement for payment table	37
6.11	Select DML statement for payment table	38
6.12	Insert DML statement for license_right table	38
6.13	Select DML statement for license_right table	39
6.14	DML statement for renewal table	39
6.15	Insert DML statement for license_history table	40
6.16	Select DML statement for license_history table	40
7.1	Join Query example 1	41
7.2	output of the 1st join query	42
7.3	Join query example 2	42
7.4	Output of 2nd join query	43
7.5	Aggregate function and GroupBy query example 1	43
7.6	Aggregate function and GroupBy query example 2	44
7.7	Aggregate function and GroupBy query example 3	44
7.8	Nested query example	45

7.9	Licensing Analysis Queries and Execution Results example 1	46
7.10	Licensing Analysis Queries and Execution Results example 2	46
8.1	Stored Procedure GetAssetLicenses and Execution Result	47
8.2	Stored Function CheckLicenseAvailability and Execution Result	48
8.3	Trigger trg_renewal_history and Execution Result	49
8.4	SQL View active_license_overview	50
8.5	SQL View license_revenue_summary	51
8.6	SQL Transaction for License Renewal	52
9.1	Application Login Page	54
9.2	System Administrator Dashboard	55
9.3	Creating a new owner in admin dashboard	55
9.4	Successful creation of OWN007 (owner ID 7)	56
9.5	License Manager Dashboard	56
9.6	Licensing Agreement Management	57
9.7	Renewal and Payment Operations	58
9.8	Digital Asset Access Simulation	59
9.9	Violation Monitoring (1st entry)	59
10.1	MongoDB Database and Collections	61
10.2	MongoDB Access Log Documents	61-62
10.3	MongoDB Violation Reports	62-63
10.4	MongoDB Read Operation	63
10.5	MongoDB Update Operation	64

10.6	**MongoDB Delete Operation**	**64**
10.7	**Violation Resolution Before and After Update**	**65**
11.1	**MySQL role creation and privileges granting**	**68**
11.2	**MySQL user creation**	**68**

1. INTRODUCTION

1.1 Background

With the increasing use of digital platforms, content such as movies, music, software, and e-books is distributed across different regions and organizations. Digital assets are often licensed to multiple companies under specific conditions, including geographical restrictions, validity periods, and permitted usage rights.

Managing these agreements requires maintaining accurate records of asset ownership, licensing details, payments, and renewals. Without a proper database system, tracking these records can become difficult and may result in licensing conflicts, unauthorized usage, or inconsistent information.

The **Digital Rights Management and Asset Licensing Ledger** addresses these challenges by providing a database-driven system for organizing licensing information and monitoring digital asset usage.

1.2 Objectives

The main objectives of the project are:

1. To develop a centralized database for managing digital assets, owners, licensees, licensing agreements, and usage rights.
2. To maintain payment records, license renewals, and licensing history.
3. To ensure data integrity using primary keys, foreign keys, and database constraints.
4. To implement advanced SQL concepts such as joins, nested queries, views, stored procedures, functions, triggers, and transactions.
5. To record digital asset access attempts and identify licensing violations using MongoDB.
6. To provide a web-based interface for performing and monitoring database operations.
7. To implement authentication and role-based access control for System Administrators and License Managers.

1.3 Implementation Overview

The project is implemented using **MySQL, MongoDB, and Next.js**.

MySQL serves as the primary relational database, storing information about digital assets, owners, licensees, territories, rights, licensing agreements, payments, renewals, and license history. It also supports advanced SQL operations and database-level access control.

MongoDB is used to store access logs and licensing violation reports. Access attempts are recorded as allowed or denied based on licensing conditions. Detected violations can be reviewed and resolved while retaining their original details.

Next.js is used to develop the web application, including the frontend interface and backend APIs. The application allows authorized users to manage licensing records, view reports, monitor activities, and interact with both databases. Authentication and role-based permissions restrict operations according to the user's responsibilities.

2. PROBLEM DEFINITION

2.1 Problem Statement

The management of digital assets and their licensing agreements involves maintaining information about ownership, licensees, geographical territories, usage rights, payments, and validity periods. As digital assets may be licensed to different organizations under varying conditions, tracking these agreements becomes increasingly complex.

Traditional or manually maintained records can lead to data duplication, inconsistent licensing information, difficulty in tracking renewals, and conflicts between licensing agreements. Furthermore, identifying unauthorized access attempts and maintaining a history of licensing violations requires a reliable monitoring mechanism.

Therefore, there is a need for a centralized database management system that can efficiently store, organize, retrieve, and manage digital licensing information while supporting access monitoring and controlled user permissions.

2.2 Proposed Solution

The proposed Digital Rights Management and Asset Licensing Ledger provides a web-based platform for managing digital assets and licensing activities.

The system uses a relational database to maintain structured licensing records and enforce relationships between entities. It supports operations such as creating and updating licensing agreements, recording payments, managing renewals, and retrieving licensing information through SQL queries.

A NoSQL database is integrated to record digital asset access attempts and licensing violations. This enables the system to maintain access history, identify violations such as territory mismatches, and allow authorized users to resolve reported incidents.

The application also provides separate access privileges for System Administrators and License Managers, ensuring that users can perform operations according to their assigned roles.

2.3 Scope of the Project

The scope of the project includes:

1. Managing digital assets, owners, licensees, territories, and usage rights.
2. Maintaining licensing agreements, payments, renewals, and license history.
3. Performing SQL operations and generating licensing-related reports.
4. Implementing database constraints and advanced SQL features.
5. Recording simulated asset access attempts and managing violation reports.
6. Providing a Next.js web interface with authentication and role-based access control.

2.4 Functional Requirements

The functional requirements describe the major operations supported by the Digital Rights Management and Asset Licensing Ledger.

Sl. No.	Functional Requirement	Description
FR1	User Authentication	Allow registered users to log in securely using their credentials.
FR2	Role-Based Access Control	Provide separate permissions for System Administrators and License Managers.
FR3	Digital Asset Management	Add, view, update, and delete digital asset records.
FR4	Licensing Agreement Management	Create, view, update, and delete licensing agreements, subject to database constraints.
FR5	Owner, Licensee, Territory, and Rights Management	Maintain information about asset owners, licensees, geographical territories, and permitted usage rights.
FR6	Payment and Renewal Management	Record licensing payments, process renewals, and maintain license history.
FR7	License Availability Checking	Check whether an asset is already licensed in a specified territory on a given date.
FR8	Licensing Queries and Reports	Retrieve information about active licenses, expirations, revenue, licensing conflicts, and related records.

FR9	Asset Access Simulation	Simulate digital asset access requests and determine whether access should be allowed or denied.
FR10	Access Logging	Store access attempt details and their outcomes in MongoDB.
FR11	Violation Detection and Resolution	Generate licensing violation reports and allow authorized users to review and resolve them.
FR12	Web-Based Interface	Provide a Next.js interface for accessing and managing database operations through backend APIs.

Table 2.1: Functional requirements of the Mini Project

3. ER MODEL

The Entity-Relationship (ER) model represents the conceptual design of the Digital Rights Management and Asset Licensing Ledger. It identifies the major entities, their attributes, and the relationships between them.

The system consists of ten main entities: **Owner, Asset, Licensee, Territory, Right Entitlement, License, License Right, Renewal, Payment, and License History**.

The **License** entity forms the central part of the model, connecting digital assets with licensees and territories. Each asset belongs to an owner, while a license specifies the organization receiving the licensing rights, the permitted territory, and the validity period.

The major relationships represented in the ER model are:

1. **Owner–Asset:** One owner can own multiple digital assets, while each asset belongs to one owner.
2. **Asset–License:** An asset can have multiple licensing agreements, while each license refers to one asset.
3. **Licensee–License:** A licensee can hold multiple licenses, while each license is associated with one licensee.
4. **Territory–License:** A territory can be associated with multiple licenses, while each license specifies one territory.
5. **License–Right Entitlement:** Licenses and rights have a many-to-many relationship, represented through the License Right entity.
6. **License–Renewal, Payment, and License History:** A license can have multiple renewal records, payment transactions, and history entries.

The ER model provides the foundation for converting the conceptual design into a relational database schema with appropriate primary keys, foreign keys, and integrity constraints.

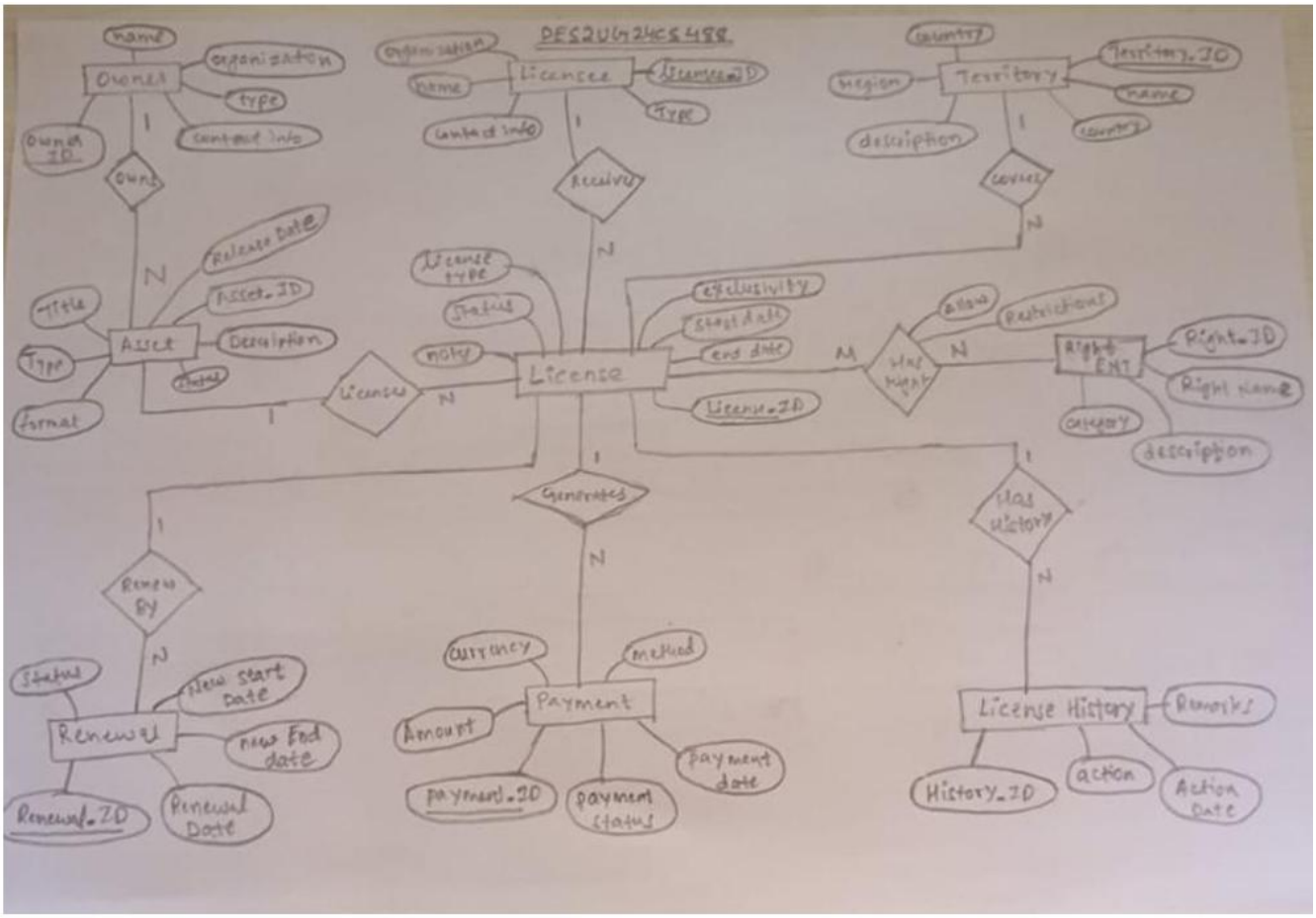


Figure 3.1: ER Diagram of Digital Rights Management and Asset Licensing Ledger

Entity	Attributes
OWNER	OwnerID (PK), Name, Organization, Type, ContactInfo
ASSET	AssetID (PK), OwnerID (FK), Title, Type, Description, Format, ReleaseDate, Status
LICENSEE	LicenseeID (PK), Name, Organization, Type, ContactInfo
TERRITORY	TerritoryID (PK), Name, Region, Country, Description
RIGHT_ENT	RightID (PK), RightName, Description, Category
LICENSE	LicenseID (PK), AssetID (FK), LicenseeID (FK), TerritoryID (FK), StartDate, EndDate, LicenseType, Exclusivity, Status, Notes
LICENSE_RIGHT	LicenseID (PK, FK), RightID (PK, FK), Allowed, Restrictions
RENEWAL	RenewalID (PK), LicenseID (FK), RenewalDate, NewStartDate, NewEndDate, Status

PAYMENT	PaymentID (PK), LicenseID (FK), Amount, Currency, PaymentStatus, PaymentDate, Method
LICENSE_HISTORY	HistoryID (PK), LicenseID (FK), Action, ActionDate, Remarks

Table 3.1 Summary of Entities and Attributes

Pls Note: PK represents Primary Key, and FK represents Foreign Key. The LICENSE_RIGHT entity has a composite primary key consisting of LicenseID and RightID.

Relationship	Cardinality	Description
OWNER — ASSET	1:N	One owner can own multiple digital assets.
ASSET — LICENSE	1:N	One asset can have multiple licensing agreements.
LICENSEE — LICENSE	1:N	One licensee can hold multiple licenses.
TERRITORY — LICENSE	1:N	One territory can be associated with multiple licenses.
LICENSE — RIGHT_ENT	M:N	A license can grant multiple rights, and a right can belong to multiple licenses.
LICENSE — RENEWAL	1:N	One license can have multiple renewal records.
LICENSE — PAYMENT	1:N	One license can have multiple payment records.
LICENSE — LICENSE_HISTORY	1:N	One license can have multiple history entries.

The many-to-many relationship between LICENSE and RIGHT_ENT is implemented through the associative entity LICENSE_RIGHT.

Table 3.2 Relationships and Cardinality Ratios

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

The Entity-Relationship (ER) model of the Digital Rights Management and Asset Licensing Ledger is converted into a relational schema by mapping entities, attributes, and relationships into tables with appropriate keys and constraints. The steps followed are:

1. **Identify Strong Entities:** Identify the main entities in the ER diagram that have their own primary keys. In our project, these include OWNER, ASSET, LICENSEE, TERRITORY, RIGHT_ENT, LICENSE, RENEWAL, PAYMENT, and LICENSE_HISTORY.
2. **Create Relations for Strong Entities:** Convert each strong entity into a separate relational table. For example, the OWNER entity is converted into the OWNER table, and the ASSET entity is converted into the ASSET table.
3. **Map Entity Attributes:** Convert the attributes of each entity into columns of its corresponding table. For example, the ASSET table contains AssetID, Title, Type, Description, Format, ReleaseDate, and Status, along with OwnerID.
4. **Assign Primary Keys:** Identify a unique attribute for each relation and designate it as the primary key. For example, OwnerID is the primary key of OWNER, AssetID is the primary key of ASSET, and LicenseID is the primary key of LICENSE.
5. **Map the OWNER–ASSET Relationship:** Convert the one-to-many (1) relationship between OWNER and ASSET by adding OwnerID as a foreign key in the ASSET table. This allows one owner to be associated with multiple assets.
6. **Map the ASSET–LICENSE Relationship:** Add AssetID as a foreign key in the LICENSE table to represent the one-to-many relationship between assets and licensing agreements.
7. **Map the LICENSEE–LICENSE Relationship:** Add LicenseeID as a foreign key in the LICENSE table. This establishes the relationship in which one licensee can hold multiple licensing agreements.
8. **Map the TERRITORY–LICENSE Relationship:** Add TerritoryID as a foreign key in the LICENSE table. This associates each licensing agreement with a particular geographical territory.

9. **Map the Many-to-Many Relationship:** The relationship between LICENSE and RIGHT_ENT is many-to-many (M), since one license can grant multiple rights and the same right can be associated with multiple licenses. Convert this relationship into a separate associative table called LICENSE_RIGHT.
10. **Define the Associative Relation:** Include LicenseID and RightID as foreign keys in LICENSE_RIGHT. Together, these attributes form a composite primary key. Additional attributes such as Allowed and Restrictions are stored in this table.
11. **Map the LICENSE–RENEWAL Relationship:** Add LicenseID as a foreign key in RENEWAL to represent the one-to-many relationship between a licensing agreement and its renewal records.
12. **Map the LICENSE–PAYMENT Relationship:** Add LicenseID as a foreign key in PAYMENT. This allows multiple payment records to be associated with a single licensing agreement.
13. **Map the LICENSE–LICENSE_HISTORY Relationship:** Add LicenseID as a foreign key in LICENSE_HISTORY to maintain multiple historical records associated with each license.
14. **Define Referential Integrity Constraints:** Establish foreign key constraints between related tables to ensure that referenced records exist. For example, a license cannot reference an AssetID that does not exist in the ASSET table.
15. **Apply Domain and Integrity Constraints:** Define suitable data types, NOT NULL constraints, and other validation rules. For example, the LICENSE table includes a CHECK constraint to ensure that EndDate is greater than or equal to StartDate.
16. **Verify the Relational Schema:** Check that all entities and relationships from the ER diagram are correctly represented, primary keys uniquely identify records, and foreign keys maintain the required relationships.
17. **Finalize the Schema:** The resulting relational schema consists of ten interconnected tables: OWNER, ASSET, LICENSEE, TERRITORY, RIGHT_ENT, LICENSE, LICENSE_RIGHT, RENEWAL, PAYMENT, and LICENSE_HISTORY. These tables form the foundation for implementing the database in MySQL.

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

The relational schema diagram represents the logical structure of the Digital Rights Management and Asset Licensing Ledger after converting the ER model into relational tables.

The schema consists of ten interconnected tables, with each table having a primary key to uniquely identify its records. Foreign keys establish relationships between tables and maintain referential integrity.

The LICENSE table serves as the central relation, connecting ASSET, LICENSEE, and TERRITORY. The LICENSE_RIGHT table resolves the many-to-many relationship between licenses and rights using a composite primary key. The RENEWAL, PAYMENT, and LICENSE_HISTORY tables are linked to LICENSE to maintain related transactions and historical records.

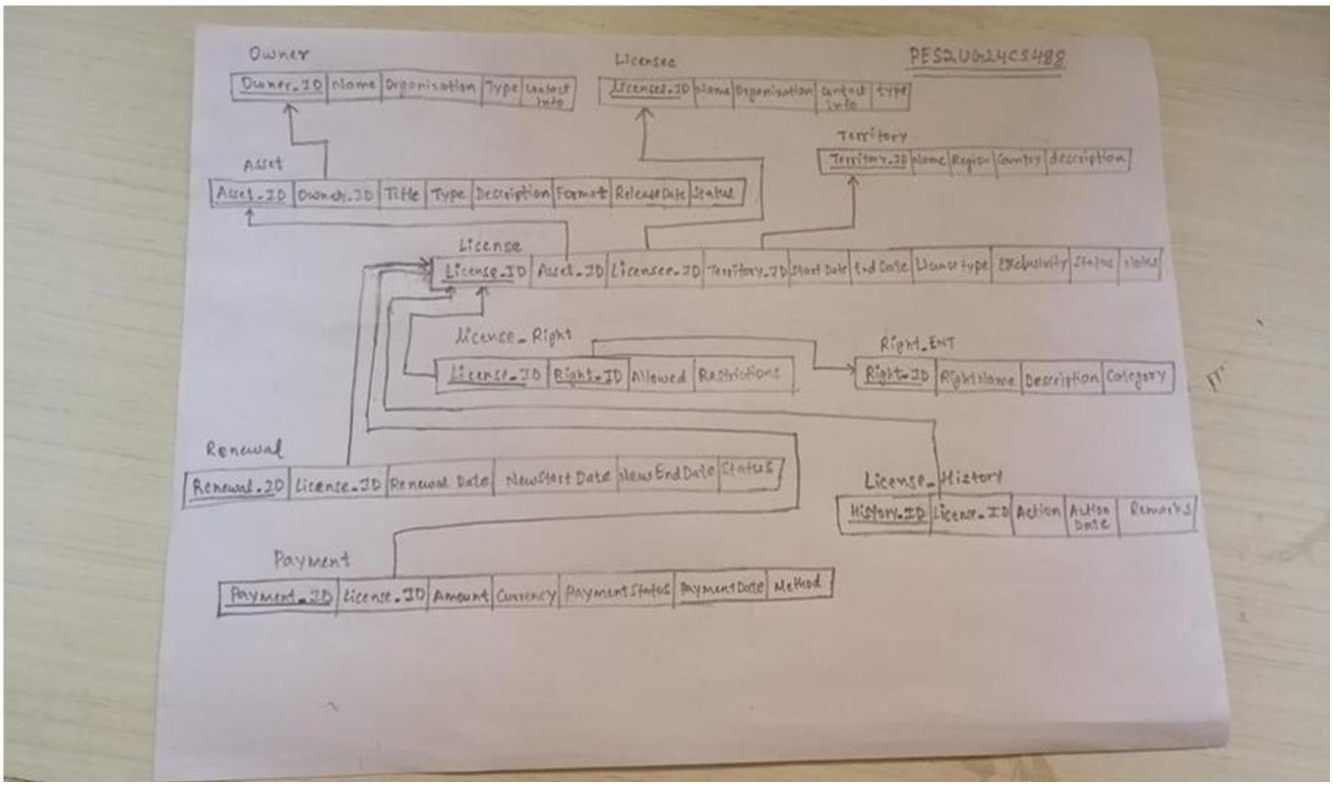


Figure 4.1: Relational Schema Diagram of Digital Rights Management and Asset Licensing Ledger

4.3 Attributes of the Relational Schema

The relational schema consists of ten tables. The attributes of each relation are listed below for reference.

1. OWNER

- OwnerID (Primary Key)
- Name
- Organization
- Type
- ContactInfo

2. ASSET

- AssetID (Primary Key)
- OwnerID (Foreign Key)
- Title
- Type
- Description
- Format
- ReleaseDate
- Status

3. LICENSEE

- LicenseeID (Primary Key)
- Name
- Organization
- Type
- ContactInfo

4. TERRITORY

- TerritoryID (Primary Key)
- Name
- Region
- Country
- Description

5. RIGHT_ENT

- RightID (Primary Key)
- RightName

- Description
- Category

6. LICENSE

- LicenseID (Primary Key)
- AssetID (Foreign Key)
- LicenseeID (Foreign Key)
- TerritoryID (Foreign Key)
- StartDate
- EndDate
- LicenseType
- Exclusivity
- Status
- Notes

7. LICENSE_RIGHT

- LicenseID (Primary Key, Foreign Key)
- RightID (Primary Key, Foreign Key)
- Allowed
- Restrictions

8. RENEWAL

- RenewalID (Primary Key)
- LicenseID (Foreign Key)
- RenewalDate
- NewStartDate
- NewEndDate
- Status

9. PAYMENT

- PaymentID (Primary Key)
- LicenseID (Foreign Key)
- Amount
- Currency
- PaymentStatus
- PaymentDate

- Method

10. LICENSE_HISTORY

- HistoryID (Primary Key)
- LicenseID (Foreign Key)
- Action
- ActionDate
- Remarks

Note: Section 4.3 is for reference in case the schema diagram is unclear PK denotes Primary Key, and FK denotes Foreign Key. The LICENSE_RIGHT table uses a composite primary key consisting of LicenseID and RightID.

5. DDL STATEMENTS

Data Definition Language (DDL) is used to define the structure of a relational database, including tables, attributes, primary keys, foreign keys, and integrity constraints.

For the Digital Rights Management and Asset Licensing Ledger, the relational database was implemented using MySQL. The database consists of ten main tables that store information about digital assets, owners, licensees, territories, licensing rights, agreements, payments, renewals, and license history.

5.1 Database Creation

The MySQL database was created with the name drm_licensing. It serves as the primary relational database for storing and managing licensing information.

```
mysql> prompt (PES2UG24CS488@\U) [\d] >\_
PROMPT set to '(PES2UG24CS488@\U) [\d] >\_'
(PES2UG24CS488@root@localhost) [(none)] > CREATE DATABASE drm_licensing;
Query OK, 1 row affected (0.03 sec)

(PES2UG24CS488@root@localhost) [(none)] > use drm_licensing;
Database changed
(PES2UG24CS488@root@localhost) [drm_licensing] >
```

Fig 5.1: DDL statement for drm_licensing database

5.2 Table Creation

The following tables were created using SQL DDL statements:

1. **OWNER:** Stores information about digital asset owners.
2. **LICENSEE:** Stores details of organizations or individuals receiving licensing rights.
3. **TERRITORY:** Stores geographical regions in which licensing rights may be granted.
4. **RIGHT_ENT:** Stores the different types of rights associated with digital assets.
5. **ASSET:** Stores digital asset details and references the corresponding owner.
6. **LICENSE:** Stores licensing agreements, including the associated asset, licensee, territory, and validity period.

7. **LICENSE_RIGHT:** Associates licensing agreements with the rights granted under each license.
8. **RENEWAL:** Stores renewal details and updated licensing periods.
9. **PAYMENT:** Maintains payment transactions associated with licensing agreements.
10. **LICENSE_HISTORY:** Records historical activities and changes associated with licenses.

```
Database changed
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE OWNER (
    ->     OwnerID VARCHAR(50) PRIMARY KEY,
    ->     Name VARCHAR(100) NOT NULL,
    ->     Organization VARCHAR(100),
    ->     Type VARCHAR(50),
    ->     ContactInfo VARCHAR(150)
    -> );
Query OK, 0 rows affected (0.08 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc owner;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| OwnerID      | varchar(50)  | NO   | PRI | NULL    |       |
| Name         | varchar(100) | NO   |     | NULL    |       |
| Organization | varchar(100) | YES  |     | NULL    |       |
| Type         | varchar(50)  | YES  |     | NULL    |       |
| ContactInfo  | varchar(150) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
5 rows in set (0.03 sec)
```

Figure 5.2: DDL statement for owner table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE LICENSEE (
    ->     LicenseeID VARCHAR(50) PRIMARY KEY,
    ->     Name VARCHAR(100) NOT NULL,
    ->     Organization VARCHAR(100),
    ->     Type VARCHAR(50),
    ->     ContactInfo VARCHAR(150)
    -> );
Query OK, 0 rows affected (0.06 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc licensee;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| LicenseeID   | varchar(50)  | NO   | PRI | NULL    |       |
| Name         | varchar(100) | NO   |     | NULL    |       |
| Organization | varchar(100) | YES  |     | NULL    |       |
| Type         | varchar(50)  | YES  |     | NULL    |       |
| ContactInfo  | varchar(150) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

Figure 5.3: DDL statement for licensee table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE TERRITORY (
    ->     TerritoryID VARCHAR(50) PRIMARY KEY,
    ->     Name VARCHAR(100) NOT NULL,
    ->     Region VARCHAR(100),
    ->     Country VARCHAR(100),
    ->     Description TEXT
    -> );
Query OK, 0 rows affected (0.06 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc territory;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| TerritoryID | varchar(50)  | NO   | PRI | NULL    |       |
| Name        | varchar(100) | NO   |     | NULL    |       |
| Region      | varchar(100) | YES  |     | NULL    |       |
| Country     | varchar(100) | YES  |     | NULL    |       |
| Description | text         | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

Fig 5.4: DDL statement for territory table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE RIGHT_ENT (
    ->     RightID VARCHAR(50) PRIMARY KEY,
    ->     RightName VARCHAR(100) NOT NULL,
    ->     Description TEXT,
    ->     Category VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.05 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc right_ent;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| RightID     | varchar(50)  | NO   | PRI | NULL    |       |
| RightName   | varchar(100) | NO   |     | NULL    |       |
| Description | text         | YES  |     | NULL    |       |
| Category    | varchar(50)  | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Fig 5.5: DDL statement for right_ent table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE ASSET (
    ->     AssetID VARCHAR(50) PRIMARY KEY,
    ->     OwnerID VARCHAR(50) NOT NULL,
    ->     Title VARCHAR(150) NOT NULL,
    ->     Type VARCHAR(50),
    ->     Description TEXT,
    ->     Format VARCHAR(50),
    ->     ReleaseDate DATE,
    ->     Status VARCHAR(50),
    ->
    ->     CONSTRAINT fk_asset_owner
    ->         FOREIGN KEY (OwnerID)
    ->         REFERENCES OWNER(OwnerID)
    ->         ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.07 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc asset;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| AssetID     | varchar(50)  | NO   | PRI | NULL    |       |
| OwnerID     | varchar(50)  | NO   | MUL | NULL    |       |
| Title       | varchar(150) | NO   |     | NULL    |       |
| Type        | varchar(50)  | YES  |     | NULL    |       |
| Description | text         | YES  |     | NULL    |       |
| Format      | varchar(50)  | YES  |     | NULL    |       |
| ReleaseDate | date         | YES  |     | NULL    |       |
| Status      | varchar(50)  | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
```

Fig 5.6: DDL statement for asset table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE LICENSE (
    ->     LicenseID VARCHAR(50) PRIMARY KEY,
    ->     AssetID VARCHAR(50) NOT NULL,
    ->     LicenseeID VARCHAR(50) NOT NULL,
    ->     TerritoryID VARCHAR(50) NOT NULL,
    ->     StartDate DATE NOT NULL,
    ->     EndDate DATE NOT NULL,
    ->     LicenseType VARCHAR(50),
    ->     Exclusivity VARCHAR(50),
    ->     Status VARCHAR(50),
    ->     Notes TEXT,
    ->
    ->     CONSTRAINT fk_license_asset
    ->         FOREIGN KEY (AssetID)
    ->         REFERENCES ASSET(AssetID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT fk_license_licensee
    ->         FOREIGN KEY (LicenseeID)
    ->         REFERENCES LICENSEE(LicenseeID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT fk_license_territory
    ->         FOREIGN KEY (TerritoryID)
    ->         REFERENCES TERRITORY(TerritoryID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT chk_license_dates
    ->         CHECK (EndDate >= StartDate)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

Fig 5.7: DDL statement for license table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE LICENSE_RIGHT (
    ->     LicenseID VARCHAR(50) NOT NULL,
    ->     RightID VARCHAR(50) NOT NULL,
    ->     Allowed BOOLEAN NOT NULL DEFAULT TRUE,
    ->     Restrictions TEXT,
    ->
    ->     PRIMARY KEY (LicenseID, RightID),
    ->
    ->     CONSTRAINT fk_license_right_license
    ->         FOREIGN KEY (LicenseID)
    ->         REFERENCES LICENSE(LicenseID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT fk_license_right_right
    ->         FOREIGN KEY (RightID)
    ->         REFERENCES RIGHT_ENT(RightID)
    ->         ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.07 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc license_right;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| LicenseID    | varchar(50) | NO   | PRI | NULL    |       |
| RightID      | varchar(50) | NO   | PRI | NULL    |       |
| Allowed      | tinyint(1)  | NO   |     | 1       |       |
| Restrictions | text        | YES  |     | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Fig 5.8: DDL statement for license_right table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE RENEWAL (
    ->     RenewalID VARCHAR(50) PRIMARY KEY,
    ->     LicenseID VARCHAR(50) NOT NULL,
    ->     RenewalDate DATE NOT NULL,
    ->     NewStartDate DATE NOT NULL,
    ->     NewEndDate DATE NOT NULL,
    ->     Status VARCHAR(50),
    ->
    ->     CONSTRAINT fk_renewal_license
    ->         FOREIGN KEY (LicenseID)
    ->         REFERENCES LICENSE(LicenseID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT chk_renewal_dates
    ->         CHECK (NewEndDate >= NewStartDate)
    -> );
Query OK, 0 rows affected (0.06 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc renewal;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| RenewalID    | varchar(50) | NO   | PRI | NULL    |       |
| LicenseID    | varchar(50) | NO   | MUL | NULL    |       |
| RenewalDate  | date        | NO   |     | NULL    |       |
| NewStartDate | date        | NO   |     | NULL    |       |
| NewEndDate   | date        | NO   |     | NULL    |       |
| Status       | varchar(50) | YES  |     | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
6 rows in set (0.03 sec)
```

Figure 5.9: DDL statement for renewal table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE PAYMENT (
    ->     PaymentID VARCHAR(50) PRIMARY KEY,
    ->     LicenseID VARCHAR(50) NOT NULL,
    ->     Amount DECIMAL(10,2) NOT NULL,
    ->     Currency VARCHAR(10) NOT NULL DEFAULT 'USD',
    ->     PaymentStatus VARCHAR(50),
    ->     PaymentDate DATE NOT NULL,
    ->     Method VARCHAR(50),
    ->
    ->     CONSTRAINT fk_payment_license
    ->         FOREIGN KEY (LicenseID)
    ->         REFERENCES LICENSE(LicenseID)
    ->         ON UPDATE CASCADE,
    ->
    ->     CONSTRAINT chk_payment_amount
    ->         CHECK (Amount >= 0)
    -> );
Query OK, 0 rows affected (0.06 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc payment;
+---------------+---------------+------+-----+---------+-------+
| Field         | Type          | Null | Key | Default | Extra |
+---------------+---------------+------+-----+---------+-------+
| PaymentID     | varchar(50)   | NO   | PRI | NULL    |       |
| LicenseID     | varchar(50)   | NO   | MUL | NULL    |       |
| Amount        | decimal(10,2) | NO   |     | NULL    |       |
| Currency      | varchar(10)   | NO   |     | USD     |       |
| PaymentStatus | varchar(50)   | YES  |     | NULL    |       |
| PaymentDate   | date          | NO   |     | NULL    |       |
| Method        | varchar(50)   | YES  |     | NULL    |       |
+---------------+---------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

Figure 5.10: DDL statement for payment table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TABLE LICENSE_HISTORY (
    ->     HistoryID VARCHAR(50) PRIMARY KEY,
    ->     LicenseID VARCHAR(50) NOT NULL,
    ->     Action VARCHAR(100) NOT NULL,
    ->     ActionDate DATETIME NOT NULL,
    ->     Remarks TEXT,
    ->
    ->     CONSTRAINT fk_history_license
    ->         FOREIGN KEY (LicenseID)
    ->         REFERENCES LICENSE(LicenseID)
    ->         ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > desc license_history;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| HistoryID  | varchar(50) | NO   | PRI | NULL    |       |
| LicenseID  | varchar(50) | NO   | MUL | NULL    |       |
| Action     | varchar(100)| NO   |     | NULL    |       |
| ActionDate | datetime    | NO   |     | NULL    |       |
| Remarks    | text        | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

Figure 5.11: DDL statement for license_history table

5.3 Database Constraints

The following constraints were implemented to maintain relational integrity:

1. **PRIMARY KEY:** Uniquely identifies each record in a table.
2. **FOREIGN KEY:** Establishes relationships between tables and ensures referential integrity.
3. **NOT NULL:** Prevents mandatory attributes from containing NULL values.
4. **UNIQUE:** Prevents duplicate values in attributes where uniqueness is required.
5. **CHECK:** Validates data according to specified conditions, such as ensuring that the license end date is not earlier than its start date.
6. **DEFAULT:** Assigns predefined values to attributes when no value is explicitly provided.

The LICENSE_RIGHT table uses a composite primary key consisting of LicenseID and RightID, preventing duplicate associations between the same license and right.

5.4 Additional Table for Authentication

An additional app_user table was introduced during application development to support user authentication and role-based access control. This table stores the information required to identify application users and distinguish between the System Administrator and License Manager roles.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SHOW CREATE TABLE app_user;
+----------+-----------------------
```

6. DML STATEMENTS

Data Manipulation Language (DML) is used to insert, retrieve, modify, and delete records stored in a relational database.

In the Digital Rights Management and Asset Licensing Ledger, SQL operations were performed on the MySQL database to manage digital assets, licensing agreements, licensees, payments, renewals, and other related information.

The INSERT INTO statement was used to populate the database tables with sample records.

The SELECT statement was used to retrieve records from the database and verify the inserted information.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO OWNER
    -> (OwnerID, Name, Organization, Type, ContactInfo)
    -> VALUES
    -> ('OWN001', 'Warner Media', 'Warner Bros. Discovery', 'Corporation', 'contact@warner.example'),
    -> ('OWN002', 'Universal Music Group', 'Universal Music Group', 'Corporation', 'licensing@umg.example'),
    -> ('OWN003', 'Adobe Systems', 'Adobe Inc.', 'Corporation', 'rights@adobe.example'),
    -> ('OWN004', 'Penguin Publishing', 'Penguin Random House', 'Publisher', 'licensing@penguin.example'),
    -> ('OWN005', 'Pixar Studios', 'Pixar Animation Studios', 'Studio', 'rights@pixar.example'),
    -> ('OWN006', 'SoundWave Records', 'SoundWave Entertainment', 'Music Label', 'legal@soundwave.example');
Query OK, 6 rows affected (0.05 sec)
Records: 6  Duplicates: 0  Warnings: 0

(PES2UG24CS488@root@localhost) [drm_licensing] > select * from owner;
+---------+----------------------+------------------------+-------------+--------------------------+
| OwnerID | Name                 | Organization           | Type        | ContactInfo              |
+---------+----------------------+------------------------+-------------+--------------------------+
| OWN001  | Warner Media         | Warner Bros. Discovery | Corporation | contact@warner.example   |
| OWN002  | Universal Music Group | Universal Music Group | Corporation | licensing@umg.example    |
| OWN003  | Adobe Systems        | Adobe Inc.             | Corporation | rights@adobe.example     |
| OWN004  | Penguin Publishing   | Penguin Random House   | Publisher   | licensing@penguin.example |
| OWN005  | Pixar Studios        | Pixar Animation Studios | Studio     | rights@pixar.example     |
| OWN006  | SoundWave Records    | SoundWave Entertainment | Music Label | legal@soundwave.example  |
+---------+----------------------+------------------------+-------------+--------------------------+
6 rows in set (0.00 sec)
```

Figure 6.1: DML statement for owner table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO LICENSEE
    -> (LicenseeID, Name, Organization, Type, ContactInfo)
    -> VALUES
    -> ('LES001', 'Netflix', 'Netflix Inc.', 'Streaming Platform', 'licensing@netflix.example'),
    -> ('LES002', 'Spotify', 'Spotify AB', 'Music Platform', 'rights@spotify.example'),
    -> ('LES003', 'TechSoft Solutions', 'TechSoft Pvt Ltd', 'Software Company', 'legal@techsoft.example'),
    -> ('LES004', 'BookWorld', 'BookWorld India', 'Publisher/Distributor', 'licenses@bookworld.example'),
    -> ('LES005', 'Amazon Prime Video', 'Amazon Inc.', 'Streaming Platform', 'content@prime.example'),
    -> ('LES006', 'MediaHub', 'MediaHub Digital', 'Digital Distributor', 'licensing@mediahub.example');
Query OK, 6 rows affected (0.03 sec)
Records: 6  Duplicates: 0  Warnings: 0

(PES2UG24CS488@root@localhost) [drm_licensing] > select * from licensee;
+------------+--------------------+-----------------+----------------------+--------------------------+
| LicenseeID | Name               | Organization    | Type                 | ContactInfo              |
+------------+--------------------+-----------------+----------------------+--------------------------+
| LES001     | Netflix            | Netflix Inc.    | Streaming Platform   | licensing@netflix.example |
| LES002     | Spotify            | Spotify AB      | Music Platform       | rights@spotify.example   |
| LES003     | TechSoft Solutions | TechSoft Pvt Ltd | Software Company    | legal@techsoft.example   |
| LES004     | BookWorld          | BookWorld India | Publisher/Distributor | licenses@bookworld.example |
| LES005     | Amazon Prime Video | Amazon Inc.     | Streaming Platform   | content@prime.example    |
| LES006     | MediaHub           | MediaHub Digital | Digital Distributor | licensing@mediahub.example |
+------------+--------------------+-----------------+----------------------+--------------------------+
6 rows in set (0.00 sec)
```

Figure 6.2: DML statement for licensee table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO TERRITORY
    -> (TerritoryID, Name, Region, Country, Description)
    -> VALUES
    -> ('TER001', 'United States', 'North America', 'USA',
    ->  'United States territory'),
    ->
    -> ('TER002', 'India', 'South Asia', 'India',
    ->  'Indian territory'),
    ->
    -> ('TER003', 'European Union', 'Europe', 'EU',
    ->  'European Union territory'),
    ->
    -> ('TER004', 'Worldwide', 'Global', 'All Countries',
    ->  'Worldwide distribution rights'),
    ->
    -> ('TER005', 'United Kingdom', 'Europe', 'UK',
    ->  'United Kingdom territory'),
    ->
    -> ('TER006', 'Australia', 'Oceania', 'Australia',
    ->  'Australian territory');
Query OK, 6 rows affected (0.03 sec)
Records: 6  Duplicates: 0  Warnings: 0

(PES2UG24CS488@root@localhost) [drm_licensing] > select * from territory;
+-------------+---------------+---------------+---------------+-------------------------------+
| TerritoryID | Name          | Region        | Country       | Description                   |
+-------------+---------------+---------------+---------------+-------------------------------+
| TER001      | United States | North America | USA           | United States territory       |
| TER002      | India         | South Asia    | India         | Indian territory              |
| TER003      | European Union | Europe       | EU            | European Union territory      |
| TER004      | Worldwide     | Global        | All Countries | Worldwide distribution rights |
| TER005      | United Kingdom | Europe       | UK            | United Kingdom territory      |
| TER006      | Australia     | Oceania       | Australia     | Australian territory          |
+-------------+---------------+---------------+---------------+-------------------------------+
6 rows in set (0.00 sec)
```

Figure 6.3: DML statement for territory table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO RIGHT_ENT
    -> (RightID, RightName, Description, Category)
    -> VALUES
    -> ('RGT001', 'Streaming',
    ->  'Right to stream digital content online',
    ->  'Distribution'),
    ->
    -> ('RGT002', 'Download',
    ->  'Right to provide downloadable digital content',
    ->  'Distribution'),
    ->
    -> ('RGT003', 'Reproduction',
    ->  'Right to reproduce or copy the asset',
    ->  'Copyright'),
    ->
    -> ('RGT004', 'Modification',
    ->  'Right to modify or adapt the asset',
    ->  'Copyright'),
    ->
    -> ('RGT005', 'Distribution',
    ->  'Right to distribute the asset',
    ->  'Distribution'),
    ->
    -> ('RGT006', 'Public Performance',
    ->  'Right to publicly perform or exhibit the asset',
    ->  'Performance'),
    ->
    -> ('RGT007', 'Commercial Use',
    ->  'Right to use the asset for commercial purposes',
    ->  'Commercial');
Query OK, 7 rows affected (0.03 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

Figure 6.4: Insert DML statement for right_ent table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from right_ent;
+---------+--------------------+-----------------------------------------------+--------------+
| RightID | RightName          | Description                                   | Category     |
+---------+--------------------+-----------------------------------------------+--------------+
| RGT001  | Streaming          | Right to stream digital content online        | Distribution |
| RGT002  | Download           | Right to provide downloadable digital content | Distribution |
| RGT003  | Reproduction       | Right to reproduce or copy the asset          | Copyright    |
| RGT004  | Modification       | Right to modify or adapt the asset            | Copyright    |
| RGT005  | Distribution       | Right to distribute the asset                 | Distribution |
| RGT006  | Public Performance | Right to publicly perform or exhibit the asset | Performance  |
| RGT007  | Commercial Use     | Right to use the asset for commercial purposes | Commercial   |
+---------+--------------------+-----------------------------------------------+--------------+
7 rows in set (0.00 sec)
```

Figure 6.5: Select DML statement for right_ent table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO ASSET
    -> (AssetID, OwnerID, Title, Type, Description, Format, ReleaseDate, Status)
    -> VALUES
    -> ('AST001', 'OWN001', 'The Last Horizon', 'Movie',
    ->  'Science fiction feature film', 'MP4', '2025-03-15', 'Active'),
    ->
    -> ('AST002', 'OWN002', 'Summer Nights', 'Song',
    ->  'Popular music track', 'MP3', '2024-06-20', 'Active'),
    ->
    -> ('AST003', 'OWN003', 'PhotoEditor Pro', 'Software',
    ->  'Professional image editing software', 'EXE', '2025-01-10', 'Active'),
    ->
    -> ('AST004', 'OWN004', 'Database Systems Handbook', 'Book',
    ->  'Reference book on database systems', 'PDF', '2024-09-01', 'Active'),
    ->
    -> ('AST005', 'OWN001', 'Digital Shadows', 'Movie',
    ->  'Crime thriller film', 'MP4', '2025-07-05', 'Active'),
    ->
    -> ('AST006', 'OWN005', 'Ocean Adventures', 'Movie',
    ->  'Animated family adventure film', 'MP4', '2025-02-12', 'Active'),
    ->
    -> ('AST007', 'OWN002', 'Midnight Drive', 'Song',
    ->  'Electronic music track', 'MP3', '2025-05-18', 'Active'),
    ->
    -> ('AST008', 'OWN003', 'DataViz Studio', 'Software',
    ->  'Data visualization software', 'EXE', '2025-04-22', 'Active'),
    ->
    -> ('AST009', 'OWN004', 'Artificial Intelligence Basics', 'Book',
    ->  'Introductory artificial intelligence textbook', 'PDF', '2025-01-25', 'Active'),
```

Figure 6.6: Insert DML statement for asset table

Pls note: The asset table has 15 records. All are shown in the select statement

```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from asset;
+---------+---------+-------------------------------+----------+-----------------------------------------------+--------+-------------+--------+
| AssetID | OwnerID | Title                         | Type     | Description                                   | Format | ReleaseDate | Status |
+---------+---------+-------------------------------+----------+-----------------------------------------------+--------+-------------+--------+
| AST001  | OWN001  | The Last Horizon              | Movie    | Science fiction feature film                  | MP4    | 2025-03-15  | Active |
| AST002  | OWN002  | Summer Nights                 | Song     | Popular music track                           | MP3    | 2024-06-20  | Active |
| AST003  | OWN003  | PhotoEditor Pro               | Software | Professional image editing software           | EXE    | 2025-01-10  | Active |
| AST004  | OWN004  | Database Systems Handbook     | Book     | Reference book on database systems            | PDF    | 2024-09-01  | Active |
| AST005  | OWN001  | Digital Shadows               | Movie    | Crime thriller film                           | MP4    | 2025-07-05  | Active |
| AST006  | OWN005  | Ocean Adventures              | Movie    | Animated family adventure film                | MP4    | 2025-02-12  | Active |
| AST007  | OWN002  | Midnight Drive                | Song     | Electronic music track                        | MP3    | 2025-05-18  | Active |
| AST008  | OWN003  | DataViz Studio                | Software | Data visualization software                   | EXE    | 2025-04-22  | Active |
| AST009  | OWN004  | Artificial Intelligence Basics | Book     | Introductory artificial intelligence textbook | PDF    | 2025-01-25  | Active |
| AST010  | OWN005  | Robot World                   | Movie    | Animated science fiction movie                | MP4    | 2024-11-10  | Active |
| AST011  | OWN006  | Echoes of Time                | Song     | Contemporary music track                      | MP3    | 2025-08-14  | Active |
| AST012  | OWN006  | City Lights                   | Song     | Urban pop music track                         | MP3    | 2024-12-05  | Active |
| AST013  | OWN001  | Hidden Truth                  | Movie    | Mystery thriller film                         | MP4    | 2025-06-30  | Active |
| AST014  | OWN004  | Cloud Computing Guide         | Book     | Guide to modern cloud computing               | PDF    | 2025-03-05  | Active |
| AST015  | OWN003  | SecureCode Toolkit            | Software | Software security analysis toolkit            | ZIP    | 2025-09-12  | Active |
+---------+---------+-------------------------------+----------+-----------------------------------------------+--------+-------------+--------+
15 rows in set (0.00 sec)
```

Figure 6.7: Select DML statement for asset table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO LICENSE
    -> (LicenseID, AssetID, LicenseeID, TerritoryID,
    ->  StartDate, EndDate, LicenseType, Exclusivity, Status, Notes)
    -> VALUES
    ->
    -> ('LIC001', 'AST001', 'LES001', 'TER001',
    ->  '2026-01-01', '2026-12-31',
    ->  'Streaming', 'Exclusive', 'Active',
    ->  'Exclusive streaming rights in USA'),
    ->
    -> ('LIC002', 'AST001', 'LES005', 'TER003',
    ->  '2026-02-01', '2026-11-30',
    ->  'Streaming', 'Non-Exclusive', 'Active',
    ->  'European streaming rights'),
    ->
    -> ('LIC003', 'AST002', 'LES002', 'TER004',
    ->  '2026-01-15', '2026-12-31',
    ->  'Music Streaming', 'Non-Exclusive', 'Active',
    ->  'Worldwide music streaming'),
    ->
    -> ('LIC004', 'AST002', 'LES006', 'TER002',
    ->  '2026-03-01', '2026-09-30',
    ->  'Music Distribution', 'Non-Exclusive', 'Expired',
    ->  'Music distribution in India'),
    ->
    -> ('LIC005', 'AST003', 'LES003', 'TER002',
    ->  '2026-01-01', '2027-02-28',
    ->  'Software', 'Exclusive', 'Active',
    ->  'Commercial software license in India'),
    ->
    -> ('LIC006', 'AST004', 'LES004', 'TER002',
    ->  '2026-01-15', '2026-12-31',
    ->  'Digital Distribution', 'Non-Exclusive', 'Active',
    ->  'Digital distribution in India'),
```

Figure 6.8: Insert DML statement for license table

Pls note: The license table has 20 records. All are shown in the select statement

LicenseID	AssetID	LicenseeID	TerritoryID	StartDate	EndDate	LicenseType	Exclusivity	Status	Notes
LIC001	AST001	LES001	TER001	2026-01-01	2026-12-31	Streaming	Exclusive	Active	Exclusive streaming rights in USA
LIC002	AST001	LES005	TER003	2026-02-01	2026-11-30	Streaming	Non-Exclusive	Active	European streaming rights
LIC003	AST002	LES002	TER004	2026-01-15	2026-12-31	Music Streaming	Non-Exclusive	Active	Worldwide music streaming
LIC004	AST002	LES006	TER002	2026-03-01	2026-09-30	Music Distribution	Non-Exclusive	Expired	Music distribution in India
LIC005	AST003	LES003	TER002	2026-01-01	2027-02-28	Software	Exclusive	Active	Commercial software license in India
LIC006	AST004	LES004	TER002	2026-01-15	2026-12-31	Digital Distribution	Non-Exclusive	Active	Digital distribution in India
LIC007	AST005	LES001	TER003	2026-04-01	2026-09-30	Streaming	Non-Exclusive	Expired	European streaming license
LIC008	AST006	LES005	TER004	2026-01-01	2027-01-31	Streaming	Exclusive	Active	Worldwide streaming rights
LIC009	AST007	LES002	TER004	2026-02-01	2027-01-31	Music Streaming	Non-Exclusive	Active	Worldwide music streaming
LIC010	AST008	LES003	TER002	2026-03-15	2027-03-14	Software	Non-Exclusive	Active	Software usage in India
LIC011	AST009	LES004	TER005	2026-01-01	2026-12-31	Digital Distribution	Exclusive	Active	UK digital distribution
LIC012	AST010	LES001	TER001	2026-05-01	2026-12-31	Streaming	Non-Exclusive	Active	USA streaming rights
LIC013	AST011	LES002	TER003	2026-04-01	2026-10-31	Music Streaming	Non-Exclusive	Active	European music streaming
LIC014	AST012	LES006	TER004	2026-01-01	2026-06-30	Music Distribution	Non-Exclusive	Expired	Worldwide distribution license
LIC015	AST013	LES005	TER006	2026-06-01	2027-05-31	Streaming	Exclusive	Active	Australia streaming rights
LIC016	AST014	LES004	TER002	2026-02-15	2027-02-14	Digital Distribution	Non-Exclusive	Active	India digital distribution
LIC017	AST015	LES003	TER004	2026-07-01	2027-06-30	Software	Non-Exclusive	Active	Worldwide commercial use
LIC018	AST001	LES006	TER005	2026-06-01	2026-12-31	Distribution	Non-Exclusive	Active	UK distribution rights
LIC019	AST006	LES001	TER001	2026-03-01	2026-12-31	Streaming	Non-Exclusive	Active	USA streaming rights
LIC020	AST013	LES006	TER004	2026-01-01	2026-04-30	Distribution	Non-Exclusive	Expired	Worldwide distribution license

20 rows in set (0.00 sec)

Figure 6.9: Select DML statement for license table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO PAYMENT
    -> (PaymentID, LicenseID, Amount, Currency, PaymentStatus, PaymentDate, Method)
    -> VALUES
    ->
    -> ('PAY001', 'LIC001', 50000.00, 'USD', 'Paid', '2026-01-05', 'Bank Transfer'),
    -> ('PAY002', 'LIC001', 10000.00, 'USD', 'Paid', '2026-06-15', 'Bank Transfer'),
    ->
    -> ('PAY003', 'LIC002', 35000.00, 'USD', 'Paid', '2026-02-03', 'Credit Card'),
    ->
    -> ('PAY004', 'LIC003', 25000.00, 'USD', 'Paid', '2026-01-18', 'Bank Transfer'),
    -> ('PAY005', 'LIC003', 5000.00, 'USD', 'Pending', '2026-07-10', 'Credit Card'),
    ->
    -> ('PAY006', 'LIC004', 12000.00, 'USD', 'Paid', '2026-03-05', 'Bank Transfer'),
    ->
    -> ('PAY007', 'LIC005', 15000.00, 'USD', 'Paid', '2026-01-03', 'Bank Transfer'),
    -> ('PAY008', 'LIC005', 5000.00, 'USD', 'Paid', '2026-07-01', 'Bank Transfer'),
    ->
    -> ('PAY009', 'LIC006', 8000.00, 'USD', 'Paid', '2026-01-18', 'Bank Transfer'),
    ->
    -> ('PAY010', 'LIC007', 30000.00, 'USD', 'Paid', '2026-04-02', 'Credit Card'),
    ->
    -> ('PAY011', 'LIC008', 75000.00, 'USD', 'Paid', '2026-01-05', 'Bank Transfer'),
    -> ('PAY012', 'LIC008', 15000.00, 'USD', 'Paid', '2026-07-05', 'Bank Transfer'),
    ->
    -> ('PAY013', 'LIC009', 22000.00, 'USD', 'Paid', '2026-02-05', 'Credit Card'),
    ->
    -> ('PAY014', 'LIC010', 18000.00, 'USD', 'Paid', '2026-03-18', 'Bank Transfer'),
    ->
    -> ('PAY015', 'LIC011', 12000.00, 'USD', 'Paid', '2026-01-05', 'Credit Card'),
```

Figure 6.10: Insert DML statement for payment table

Pls note: Payment table has 25 records shown in the select command

```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from payment;
+-----------+-----------+----------+----------+---------------+-------------+----------------+
| PaymentID | LicenseID | Amount   | Currency | PaymentStatus | PaymentDate | Method         |
+-----------+-----------+----------+----------+---------------+-------------+----------------+
| PAY001    | LIC001    | 50000.00 | USD      | Paid          | 2026-01-05  | Bank Transfer  |
| PAY002    | LIC001    | 10000.00 | USD      | Paid          | 2026-06-15  | Bank Transfer  |
| PAY003    | LIC002    | 35000.00 | USD      | Paid          | 2026-02-03  | Credit Card    |
| PAY004    | LIC003    | 25000.00 | USD      | Paid          | 2026-01-18  | Bank Transfer  |
| PAY005    | LIC003    |  5000.00 | USD      | Pending       | 2026-07-10  | Credit Card    |
| PAY006    | LIC004    | 12000.00 | USD      | Paid          | 2026-03-05  | Bank Transfer  |
| PAY007    | LIC005    | 15000.00 | USD      | Paid          | 2026-01-03  | Bank Transfer  |
| PAY008    | LIC005    |  5000.00 | USD      | Paid          | 2026-07-01  | Bank Transfer  |
| PAY009    | LIC006    |  8000.00 | USD      | Paid          | 2026-01-18  | Bank Transfer  |
| PAY010    | LIC007    | 30000.00 | USD      | Paid          | 2026-04-02  | Credit Card    |
| PAY011    | LIC008    | 75000.00 | USD      | Paid          | 2026-01-05  | Bank Transfer  |
| PAY012    | LIC008    | 15000.00 | USD      | Paid          | 2026-07-05  | Bank Transfer  |
| PAY013    | LIC009    | 22000.00 | USD      | Paid          | 2026-02-05  | Credit Card    |
| PAY014    | LIC010    | 18000.00 | USD      | Paid          | 2026-03-18  | Bank Transfer  |
| PAY015    | LIC011    | 12000.00 | USD      | Paid          | 2026-01-05  | Credit Card    |
| PAY016    | LIC012    | 28000.00 | USD      | Paid          | 2026-05-05  | Bank Transfer  |
| PAY017    | LIC013    | 16000.00 | USD      | Paid          | 2026-04-05  | Credit Card    |
| PAY018    | LIC014    | 10000.00 | USD      | Paid          | 2026-01-05  | Bank Transfer  |
| PAY019    | LIC015    | 45000.00 | USD      | Paid          | 2026-06-05  | Bank Transfer  |
| PAY020    | LIC015    |  5000.00 | USD      | Pending       | 2026-09-05  | Credit Card    |
| PAY021    | LIC016    |  9000.00 | USD      | Paid          | 2026-02-18  | Bank Transfer  |
| PAY022    | LIC017    | 30000.00 | USD      | Paid          | 2026-07-05  | Bank Transfer  |
| PAY023    | LIC018    | 14000.00 | USD      | Paid          | 2026-06-05  | Credit Card    |
| PAY024    | LIC019    | 20000.00 | USD      | Paid          | 2026-03-05  | Bank Transfer  |
| PAY025    | LIC020    | 18000.00 | USD      | Paid          | 2026-01-05  | Credit Card    |
+-----------+-----------+----------+----------+---------------+-------------+----------------+
25 rows in set (0.00 sec)
```

Figure 6.11: Select DML statement for payment table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO LICENSE_RIGHT
    -> (LicenseID, RightID, Allowed, Restrictions)
    -> VALUES
    ->
    -> ('LIC001', 'RGT001', TRUE, 'Streaming allowed only within USA'),
    -> ('LIC001', 'RGT006', TRUE, 'Public exhibition subject to local regulations'),
    ->
    -> ('LIC002', 'RGT001', TRUE, 'Streaming allowed within EU'),
    -> ('LIC002', 'RGT002', FALSE, 'Downloads not permitted'),
    ->
    -> ('LIC003', 'RGT001', TRUE, 'Audio streaming worldwide'),
    -> ('LIC003', 'RGT002', TRUE, 'Downloads allowed for premium users'),
    ->
    -> ('LIC004', 'RGT001', TRUE, 'Streaming within India'),
    -> ('LIC004', 'RGT005', TRUE, 'Digital distribution in India'),
    ->
    -> ('LIC005', 'RGT003', TRUE, 'One authorized copy per installation'),
    -> ('LIC005', 'RGT004', FALSE, 'Modification of software prohibited'),
    -> ('LIC005', 'RGT007', TRUE, 'Commercial use allowed for registered businesses'),
    ->
    -> ('LIC006', 'RGT002', TRUE, 'PDF download allowed'),
    -> ('LIC006', 'RGT005', TRUE, 'Digital distribution within India'),
    ->
    -> ('LIC007', 'RGT001', TRUE, 'Streaming allowed during license period'),
    ->
    -> ('LIC008', 'RGT001', TRUE, 'Worldwide streaming'),
    -> ('LIC008', 'RGT002', TRUE, 'Offline download permitted'),
    ->
    -> ('LIC009', 'RGT001', TRUE, 'Worldwide audio streaming'),
    ->
    -> ('LIC010', 'RGT003', TRUE, 'Authorized installation only'),
    -> ('LIC010', 'RGT007', TRUE, 'Commercial use permitted'),
```

Figure 6.12: Insert DML statement for license_right table

Pls note: License_right table has 35 records. All of them are shown in the select command

```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from license_right;
+-----------+---------+---------+-----------------------------------------------+
| LicenseID | RightID | Allowed | Restrictions                                  |
+-----------+---------+---------+-----------------------------------------------+
| LIC001    | RGT001  |       1 | Streaming allowed only within USA             |
| LIC001    | RGT006  |       1 | Public exhibition subject to local regulations|
| LIC002    | RGT001  |       1 | Streaming allowed within EU                   |
| LIC002    | RGT002  |       0 | Downloads not permitted                       |
| LIC003    | RGT001  |       1 | Audio streaming worldwide                     |
| LIC003    | RGT002  |       1 | Downloads allowed for premium users           |
| LIC004    | RGT001  |       1 | Streaming within India                        |
| LIC004    | RGT005  |       1 | Digital distribution in India                 |
| LIC005    | RGT003  |       1 | One authorized copy per installation          |
| LIC005    | RGT004  |       0 | Modification of software prohibited           |
| LIC005    | RGT007  |       1 | Commercial use allowed for registered businesses |
| LIC006    | RGT002  |       1 | PDF download allowed                          |
| LIC006    | RGT005  |       1 | Digital distribution within India             |
| LIC007    | RGT001  |       1 | Streaming allowed during license period       |
| LIC008    | RGT001  |       1 | Worldwide streaming                           |
| LIC008    | RGT002  |       1 | Offline download permitted                    |
| LIC009    | RGT001  |       1 | Worldwide audio streaming                     |
| LIC010    | RGT003  |       1 | Authorized installation only                  |
| LIC010    | RGT007  |       1 | Commercial use permitted                      |
| LIC011    | RGT002  |       1 | Digital download in UK                        |
| LIC011    | RGT005  |       1 | Distribution restricted to UK                 |
| LIC012    | RGT001  |       1 | Streaming in USA                              |
| LIC013    | RGT001  |       1 | European streaming rights                     |
| LIC013    | RGT002  |       1 | Downloads allowed                             |
| LIC014    | RGT005  |       1 | Worldwide distribution during license period  |
| LIC015    | RGT001  |       1 | Streaming in Australia                        |
| LIC015    | RGT006  |       1 | Public exhibition allowed                     |
| LIC016    | RGT002  |       1 | Downloads within India                        |
| LIC016    | RGT005  |       1 | Digital distribution within India             |
| LIC017    | RGT003  |       1 | Authorized copies only                        |
| LIC017    | RGT004  |       0 | Modification prohibited                       |
| LIC017    | RGT007  |       1 | Commercial use permitted                      |
| LIC018    | RGT005  |       1 | Distribution limited to UK                    |
| LIC019    | RGT001  |       1 | Streaming in USA                              |
| LIC020    | RGT005  |       1 | Worldwide distribution during license period  |
+-----------+---------+---------+-----------------------------------------------+
35 rows in set (0.00 sec)
```

Figure 6.13: Select DML statement for license_right table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO RENEWAL
    -> (RenewalID, LicenseID, RenewalDate, NewStartDate, NewEndDate, Status)
    -> VALUES
    ->
    -> ('REN001', 'LIC001', '2026-11-15',
    ->  '2027-01-01', '2027-12-31', 'Approved'),
    ->
    -> ('REN002', 'LIC003', '2026-11-20',
    ->  '2027-01-15', '2028-01-14', 'Pending'),
    ->
    -> ('REN003', 'LIC005', '2026-12-01',
    ->  '2027-03-01', '2028-02-29', 'Approved'),
    ->
    -> ('REN004', 'LIC006', '2026-12-10',
    ->  '2027-01-15', '2027-12-31', 'Pending'),
    ->
    -> ('REN005', 'LIC008', '2026-12-05',
    ->  '2027-02-01', '2028-01-31', 'Approved'),
    ->
    -> ('REN006', 'LIC009', '2026-12-12',
    ->  '2027-02-01', '2028-01-31', 'Pending'),
    ->
    -> ('REN007', 'LIC011', '2026-11-25',
    ->  '2027-01-01', '2027-12-31', 'Approved'),
    ->
    -> ('REN008', 'LIC015', '2027-04-15',
    ->  '2027-06-01', '2028-05-31', 'Pending');
Query OK, 8 rows affected (0.02 sec)
Records: 8  Duplicates: 0  Warnings: 0

(PES2UG24CS488@root@localhost) [drm_licensing] > select * from renewal;
+-----------+-----------+-------------+--------------+------------+----------+
| RenewalID | LicenseID | RenewalDate | NewStartDate | NewEndDate | Status   |
+-----------+-----------+-------------+--------------+------------+----------+
| REN001    | LIC001    | 2026-11-15  | 2027-01-01   | 2027-12-31 | Approved |
| REN002    | LIC003    | 2026-11-20  | 2027-01-15   | 2028-01-14 | Pending  |
| REN003    | LIC005    | 2026-12-01  | 2027-03-01   | 2028-02-29 | Approved |
| REN004    | LIC006    | 2026-12-10  | 2027-01-15   | 2027-12-31 | Pending  |
| REN005    | LIC008    | 2026-12-05  | 2027-02-01   | 2028-01-31 | Approved |
| REN006    | LIC009    | 2026-12-12  | 2027-02-01   | 2028-01-31 | Pending  |
| REN007    | LIC011    | 2026-11-25  | 2027-01-01   | 2027-12-31 | Approved |
| REN008    | LIC015    | 2027-04-15  | 2027-06-01   | 2028-05-31 | Pending  |
+-----------+-----------+-------------+--------------+------------+----------+
8 rows in set (0.00 sec)
```

Figure 6.14: DML statement for renewal table

Pls note: Licesence_history table consists of 30 records. All of them are shown in select command

```
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO LICENSE_HISTORY
    -> (HistoryID, LicenseID, Action, ActionDate, Remarks)
    -> VALUES
    ->
    -> ('HIS001', 'LIC001', 'License Created',
    ->  '2026-01-01 10:00:00', 'Initial streaming license created'),
    ->
    -> ('HIS002', 'LIC001', 'Payment Received',
    ->  '2026-01-05 11:00:00', 'Initial license payment received'),
    ->
    -> ('HIS003', 'LIC002', 'License Created',
    ->  '2026-02-01 10:30:00', 'European streaming license created'),
    ->
    -> ('HIS004', 'LIC003', 'License Created',
    ->  '2026-01-15 09:00:00', 'Worldwide music license created'),
    ->
    -> ('HIS005', 'LIC003', 'Renewal Requested',
    ->  '2026-11-20 14:00:00', 'Renewal request submitted'),
    ->
    -> ('HIS006', 'LIC004', 'License Created',
    ->  '2026-03-01 12:00:00', 'India distribution license created'),
    ->
    -> ('HIS007', 'LIC004', 'License Expired',
    ->  '2026-10-01 00:00:00', 'License period ended'),
    ->
    -> ('HIS008', 'LIC005', 'License Created',
    ->  '2026-01-01 09:30:00', 'Software license activated'),
    ->
    -> ('HIS009', 'LIC005', 'Payment Received',
    ->  '2026-01-03 10:00:00', 'Initial payment received'),
    ->
```

Figure 6.15: Insert DML statement for license_history table

```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from license_history;
+-----------+-----------+-------------------+---------------------+----------------------------------------+
| HistoryID | LicenseID | Action            | ActionDate          | Remarks                                |
+-----------+-----------+-------------------+---------------------+----------------------------------------+
| HIS001    | LIC001    | License Created   | 2026-01-01 10:00:00 | Initial streaming license created      |
| HIS002    | LIC001    | Payment Received  | 2026-01-05 11:00:00 | Initial license payment received       |
| HIS003    | LIC002    | License Created   | 2026-02-01 10:30:00 | European streaming license created     |
| HIS004    | LIC003    | License Created   | 2026-01-15 09:00:00 | Worldwide music license created        |
| HIS005    | LIC003    | Renewal Requested | 2026-11-20 14:00:00 | Renewal request submitted              |
| HIS006    | LIC004    | License Created   | 2026-03-01 12:00:00 | India distribution license created     |
| HIS007    | LIC004    | License Expired   | 2026-10-01 00:00:00 | License period ended                   |
| HIS008    | LIC005    | License Created   | 2026-01-01 09:30:00 | Software license activated             |
| HIS009    | LIC005    | Payment Received  | 2026-01-03 10:00:00 | Initial payment received               |
| HIS010    | LIC006    | License Created   | 2026-01-15 12:00:00 | Digital distribution license created   |
| HIS011    | LIC006    | Renewal Requested | 2026-12-10 13:00:00 | Renewal request submitted              |
| HIS012    | LIC007    | License Created   | 2026-04-01 10:00:00 | European streaming license created     |
| HIS013    | LIC007    | License Expired   | 2026-10-01 00:00:00 | License period ended                   |
| HIS014    | LIC008    | License Created   | 2026-01-01 11:00:00 | Worldwide streaming license created    |
| HIS015    | LIC008    | Renewal Requested | 2026-12-05 10:00:00 | Renewal approved                       |
| HIS016    | LIC009    | License Created   | 2026-02-01 10:00:00 | Worldwide music streaming license      |
| HIS017    | LIC009    | Renewal Requested | 2026-12-12 15:00:00 | Renewal request submitted              |
| HIS018    | LIC010    | License Created   | 2026-03-15 09:00:00 | Software license activated             |
| HIS019    | LIC011    | License Created   | 2026-01-01 08:30:00 | UK digital distribution license        |
| HIS020    | LIC011    | Renewal Requested | 2026-11-25 11:00:00 | Renewal approved                       |
| HIS021    | LIC012    | License Created   | 2026-05-01 10:00:00 | USA streaming license created          |
| HIS022    | LIC013    | License Created   | 2026-04-01 10:00:00 | European music license created         |
| HIS023    | LIC014    | License Created   | 2026-01-01 09:00:00 | Worldwide distribution license         |
| HIS024    | LIC014    | License Expired   | 2026-07-01 00:00:00 | License period ended                   |
| HIS025    | LIC015    | License Created   | 2026-06-01 10:00:00 | Australia streaming license created    |
| HIS026    | LIC016    | License Created   | 2026-02-15 11:00:00 | India digital distribution license     |
| HIS027    | LIC017    | License Created   | 2026-07-01 09:00:00 | Worldwide software license created     |
| HIS028    | LIC018    | License Created   | 2026-06-01 12:00:00 | UK distribution license created        |
| HIS029    | LIC019    | License Created   | 2026-03-01 10:30:00 | USA streaming license created          |
| HIS030    | LIC020    | License Expired   | 2026-05-01 00:00:00 | Worldwide distribution license expired |
+-----------+-----------+-------------------+---------------------+----------------------------------------+
30 rows in set (0.00 sec)
```

Figure 6.16: Select DML statement for license_history table

7. QUERIES

SQL queries were implemented to retrieve, filter, and analyze information stored in the MySQL database. These queries help manage licensing agreements, track payments, identify expiring licenses, and examine relationships between digital assets and their associated rights.

7.1 Join Queries

JOIN operations were used to retrieve related information from multiple tables using their primary key and foreign key relationships.

In this project, joins were used to combine information from the ASSET, LICENSE, LICENSEE, TERRITORY, PAYMENT, and other related tables.

These queries provide consolidated licensing information, such as the asset title, licensee details, permitted territory, and associated licensing agreement.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT *
    -> FROM PAYMENT
    -> WHERE PaymentID = 'PAY025';
Empty set (0.01 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     L.LicenseID,
    ->     A.Title AS AssetTitle,
    ->     O.Name AS Owner,
    ->     LE.Name AS Licensee,
    ->     T.Name AS Territory,
    ->     L.StartDate,
    ->     L.EndDate,
    ->     L.LicenseType,
    ->     L.Exclusivity,
    ->     L.Status
    -> FROM LICENSE L
    -> JOIN ASSET A
    ->     ON L.AssetID = A.AssetID
    -> JOIN OWNER O
    ->     ON A.OwnerID = O.OwnerID
    -> JOIN LICENSEE LE
    ->     ON L.LicenseeID = LE.LicenseeID
    -> JOIN TERRITORY T
    ->     ON L.TerritoryID = T.TerritoryID
    -> ORDER BY L.EndDate ASC;
```

Figure 7.1: Join Query example 1

LicenseID	AssetTitle	Owner	Licensee	Territory	StartDate	EndDate	LicenseType	Exclusivity	Status
LIC020	Hidden Truth	Warner Media	MediaHub	Worldwide	2026-01-01	2026-04-30	Distribution	Non-Exclusive	Expired
LIC014	City Lights	SoundWave Records	MediaHub	Worldwide	2026-01-01	2026-06-30	Music Distribution	Non-Exclusive	Expired
LIC007	Digital Shadows	Warner Media	Netflix	European Union	2026-04-01	2026-09-30	Streaming	Non-Exclusive	Expired
LIC004	Summer Nights	Universal Music Group	MediaHub	India	2026-03-01	2026-09-30	Music Distribution	Non-Exclusive	Expired
LIC013	Echoes of Time	SoundWave Records	Spotify	European Union	2026-04-01	2026-10-31	Music Streaming	Non-Exclusive	Active
LIC002	The Last Horizon	Warner Media	Amazon Prime Video	European Union	2026-02-01	2026-11-30	Streaming	Non-Exclusive	Active
LIC001	The Last Horizon	Warner Media	Netflix	United States	2026-01-01	2026-12-31	Streaming	Exclusive	Active
LIC018	The Last Horizon	Warner Media	MediaHub	United Kingdom	2026-06-01	2026-12-31	Distribution	Non-Exclusive	Active
LIC003	Summer Nights	Universal Music Group	Spotify	Worldwide	2026-01-15	2026-12-31	Music Streaming	Non-Exclusive	Active
LIC006	Database Systems Handbook	Penguin Publishing	BookWorld	India	2026-01-15	2026-12-31	Digital Distribution	Non-Exclusive	Active
LIC011	Artificial Intelligence Basics	Penguin Publishing	BookWorld	United Kingdom	2026-01-01	2026-12-31	Digital Distribution	Exclusive	Active
LIC019	Ocean Adventures	Pixar Studios	Netflix	United States	2026-03-01	2026-12-31	Streaming	Non-Exclusive	Active
LIC012	Robot World	Pixar Studios	Netflix	United States	2026-05-01	2026-12-31	Streaming	Non-Exclusive	Active
LIC009	Midnight Drive	Universal Music Group	Spotify	Worldwide	2026-02-01	2027-01-31	Music Streaming	Non-Exclusive	Active
LIC008	Ocean Adventures	Pixar Studios	Amazon Prime Video	Worldwide	2026-01-01	2027-01-31	Streaming	Exclusive	Active
LIC016	Cloud Computing Guide	Penguin Publishing	BookWorld	India	2026-02-15	2027-02-14	Digital Distribution	Non-Exclusive	Active
LIC005	PhotoEditor Pro	Adobe Systems	TechSoft Solutions	India	2026-01-01	2027-02-28	Software	Exclusive	Active
LIC010	DataViz Studio	Adobe Systems	TechSoft Solutions	India	2026-03-15	2027-03-14	Software	Non-Exclusive	Active
LIC015	Hidden Truth	Warner Media	Amazon Prime Video	Australia	2026-06-01	2027-05-31	Streaming	Exclusive	Active
LIC017	SecureCode Toolkit	Adobe Systems	TechSoft Solutions	Worldwide	2026-07-01	2027-06-30	Software	Non-Exclusive	Active

20 rows in set (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >

Figure 7.2: output of the 1st join query

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     L.LicenseID,
    ->     A.Title AS AssetTitle,
    ->     LE.Name AS Licensee,
    ->     R.RightName,
    ->     LR.Allowed,
    ->     LR.Restrictions
    -> FROM LICENSE L
    -> JOIN ASSET A
    ->     ON L.AssetID = A.AssetID
    -> JOIN LICENSEE LE
    ->     ON L.LicenseeID = LE.LicenseeID
    -> JOIN LICENSE_RIGHT LR
    ->     ON L.LicenseID = LR.LicenseID
    -> JOIN RIGHT_ENT R
    ->     ON LR.RightID = R.RightID
    -> ORDER BY L.LicenseID, R.RightName;
```

Figure 7.3: Join query example 2

LicenseID	AssetTitle	Licensee	RightName	Allowed	Restrictions
LIC001	The Last Horizon	Netflix	Public Performance	1	Public exhibition subject to local regulations
LIC001	The Last Horizon	Netflix	Streaming	1	Streaming allowed only within USA
LIC002	The Last Horizon	Amazon Prime Video	Download	0	Downloads not permitted
LIC002	The Last Horizon	Amazon Prime Video	Streaming	1	Streaming allowed within EU
LIC003	Summer Nights	Spotify	Download	1	Downloads allowed for premium users
LIC003	Summer Nights	Spotify	Streaming	1	Audio streaming worldwide
LIC004	Summer Nights	MediaHub	Distribution	1	Digital distribution in India
LIC004	Summer Nights	MediaHub	Streaming	1	Streaming within India
LIC005	PhotoEditor Pro	TechSoft Solutions	Commercial Use	1	Commercial use allowed for registered businesses
LIC005	PhotoEditor Pro	TechSoft Solutions	Modification	0	Modification of software prohibited
LIC005	PhotoEditor Pro	TechSoft Solutions	Reproduction	1	One authorized copy per installation
LIC006	Database Systems Handbook	BookWorld	Distribution	1	Digital distribution within India
LIC006	Database Systems Handbook	BookWorld	Download	1	PDF download allowed
LIC007	Digital Shadows	Netflix	Streaming	1	Streaming allowed during license period
LIC008	Ocean Adventures	Amazon Prime Video	Download	1	Offline download permitted
LIC008	Ocean Adventures	Amazon Prime Video	Streaming	1	Worldwide streaming
LIC009	Midnight Drive	Spotify	Streaming	1	Worldwide audio streaming
LIC010	DataViz Studio	TechSoft Solutions	Commercial Use	1	Commercial use permitted
LIC010	DataViz Studio	TechSoft Solutions	Reproduction	1	Authorized installation only
LIC011	Artificial Intelligence Basics	BookWorld	Distribution	1	Distribution restricted to UK
LIC011	Artificial Intelligence Basics	BookWorld	Download	1	Digital download in UK
LIC012	Robot World	Netflix	Streaming	1	Streaming in USA
LIC013	Echoes of Time	Spotify	Download	1	Downloads allowed
LIC013	Echoes of Time	Spotify	Streaming	1	European streaming rights
LIC014	City Lights	MediaHub	Distribution	1	Worldwide distribution during license period
LIC015	Hidden Truth	Amazon Prime Video	Public Performance	1	Public exhibition allowed
LIC015	Hidden Truth	Amazon Prime Video	Streaming	1	Streaming in Australia
LIC016	Cloud Computing Guide	BookWorld	Distribution	1	Digital distribution within India
LIC016	Cloud Computing Guide	BookWorld	Download	1	Downloads within India
LIC017	SecureCode Toolkit	TechSoft Solutions	Commercial Use	1	Commercial use permitted
LIC017	SecureCode Toolkit	TechSoft Solutions	Modification	0	Modification prohibited
LIC017	SecureCode Toolkit	TechSoft Solutions	Reproduction	1	Authorized copies only
LIC018	The Last Horizon	MediaHub	Distribution	1	Distribution limited to UK
LIC019	Ocean Adventures	Netflix	Streaming	1	Streaming in USA
LIC020	Hidden Truth	MediaHub	Distribution	1	Worldwide distribution during license period

35 rows in set (0.02 sec)

Figure 7.4: Output of 2nd join query

7.2 Aggregate Functions and GROUP BY Queries

Aggregate functions such as COUNT(), SUM(), and AVG() were used to perform calculations on database records.

The GROUP BY clause was used to organize records into groups for analysis, while the HAVING clause was used to filter groups based on aggregate conditions.

These queries were used to analyze licensing revenue, count licensing agreements, and summarize payment information.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     O.Name AS Owner,
    ->     COUNT(L.LicenseID) AS TotalLicenses,
    ->     COUNT(DISTINCT L.AssetID) AS AssetsLicensed,
    ->     COALESCE(SUM(P.Amount), 0) AS TotalRevenue
    -> FROM OWNER O
    -> JOIN ASSET A
    ->     ON O.OwnerID = A.OwnerID
    -> JOIN LICENSE L
    ->     ON A.AssetID = L.AssetID
    -> LEFT JOIN PAYMENT P
    ->     ON L.LicenseID = P.LicenseID
    -> GROUP BY O.OwnerID, O.Name
    -> ORDER BY TotalRevenue DESC;
+-----------------------+---------------+-----------------+--------------+
| Owner                 | TotalLicenses | AssetsLicensed | TotalRevenue |
+-----------------------+---------------+-----------------+--------------+
| Warner Media          |             8 |               3 |    189000.00 |
| Pixar Studios         |             4 |               2 |    138000.00 |
| Adobe Systems         |             4 |               3 |     68000.00 |
| Universal Music Group |             4 |               2 |     64000.00 |
| Penguin Publishing    |             3 |               3 |     29000.00 |
| SoundWave Records     |             2 |               2 |     26000.00 |
+-----------------------+---------------+-----------------+--------------+
6 rows in set (0.01 sec)
```

Figure 7.5: Aggregate function and GroupBy query example 1

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     O.Name AS Owner,
    ->     SUM(P.Amount) AS TotalRevenue
    -> FROM OWNER O
    -> JOIN ASSET A
    ->     ON O.OwnerID = A.OwnerID
    -> JOIN LICENSE L
    ->     ON A.AssetID = L.AssetID
    -> JOIN PAYMENT P
    ->     ON L.LicenseID = P.LicenseID
    -> GROUP BY O.OwnerID, O.Name
    -> HAVING SUM(P.Amount) > 50000
    -> ORDER BY TotalRevenue DESC;
+----------------------+--------------+
| Owner                | TotalRevenue |
+----------------------+--------------+
| Warner Media         |    189000.00 |
| Pixar Studios        |    138000.00 |
| Adobe Systems        |     68000.00 |
| Universal Music Group |     64000.00 |
+----------------------+--------------+
4 rows in set (0.00 sec)
```

Figure 7.6: Aggregate function and GroupBy query example 2

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     A.AssetID,
    ->     A.Title,
    ->     COUNT(L.LicenseID) AS NumberOfLicenses
    -> FROM ASSET A
    -> JOIN LICENSE L
    ->     ON A.AssetID = L.AssetID
    -> GROUP BY A.AssetID, A.Title
    -> HAVING COUNT(L.LicenseID) > 1
    -> ORDER BY NumberOfLicenses DESC;
+---------+-----------------+-----------------+
| AssetID | Title           | NumberOfLicenses |
+---------+-----------------+-----------------+
| AST001  | The Last Horizon |               3 |
| AST002  | Summer Nights    |               2 |
| AST006  | Ocean Adventures |               2 |
| AST013  | Hidden Truth     |               2 |
+---------+-----------------+-----------------+
4 rows in set (0.00 sec)
```

Figure 7.7: Aggregate function and GroupBy query example 3

7.3 Nested Queries

Nested queries were used to retrieve information based on the results of another SQL query.

For example, a nested query was implemented to identify payment records with amounts greater than the average payment amount.

This demonstrates how subqueries can be used to perform comparisons and retrieve records satisfying specific conditions.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     LE.Name AS Licensee,
    ->     SUM(P.Amount) AS TotalPayment
    -> FROM LICENSEE LE
    -> JOIN LICENSE L
    ->     ON LE.LicenseeID = L.LicenseeID
    -> JOIN PAYMENT P
    ->     ON L.LicenseID = P.LicenseID
    -> GROUP BY LE.LicenseeID, LE.Name
    -> HAVING SUM(P.Amount) >
    -> (
    ->     SELECT AVG(TotalPayment)
    ->     FROM
    ->     (
    ->         SELECT
    ->             L2.LicenseeID,
    ->             SUM(P2.Amount) AS TotalPayment
    ->         FROM LICENSE L2
    ->         JOIN PAYMENT P2
    ->             ON L2.LicenseID = P2.LicenseID
    ->         GROUP BY L2.LicenseeID
    ->     ) AS LicenseeTotals
    -> )
    -> ORDER BY TotalPayment DESC;
+--------------------+--------------+
| Licensee           | TotalPayment |
+--------------------+--------------+
| Amazon Prime Video |    175000.00 |
| Netflix            |    138000.00 |
+--------------------+--------------+
2 rows in set (0.01 sec)
```

Figure 7.8: Nested query example

7.4 Licensing Analysis Queries

Additional SQL queries were implemented to support important licensing operations, including:

1. Identifying licenses approaching their expiry dates.
2. Retrieving rights associated with licensing agreements.
3. Detecting overlapping exclusive licensing agreements.
4. Identifying digital assets associated with multiple licenses.

5. Finding licensing agreements without corresponding payment records.

These queries help monitor licensing activities, identify potential conflicts, and support database reporting.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     L.LicenseID,
    ->     A.Title AS AssetTitle,
    ->     LE.Name AS Licensee,
    ->     L.EndDate,
    ->     L.Exclusivity,
    ->     L.Status
    -> FROM LICENSE L
    -> JOIN ASSET A
    ->     ON L.AssetID = A.AssetID
    -> JOIN LICENSEE LE
    ->     ON L.LicenseeID = LE.LicenseeID
    -> WHERE L.EndDate BETWEEN '2026-10-01' AND '2026-12-31'
    ->   AND L.Status = 'Active'
    -> ORDER BY L.EndDate ASC;
+-----------+--------------------------------+-------------------+------------+--------------+--------+
| LicenseID | AssetTitle                     | Licensee          | EndDate    | Exclusivity  | Status |
+-----------+--------------------------------+-------------------+------------+--------------+--------+
| LIC013    | Echoes of Time                 | Spotify           | 2026-10-31 | Non-Exclusive | Active |
| LIC002    | The Last Horizon               | Amazon Prime Video | 2026-11-30 | Non-Exclusive | Active |
| LIC001    | The Last Horizon               | Netflix           | 2026-12-31 | Exclusive     | Active |
| LIC003    | Summer Nights                  | Spotify           | 2026-12-31 | Non-Exclusive | Active |
| LIC006    | Database Systems Handbook      | BookWorld         | 2026-12-31 | Non-Exclusive | Active |
| LIC011    | Artificial Intelligence Basics | BookWorld         | 2026-12-31 | Exclusive     | Active |
| LIC012    | Robot World                    | Netflix           | 2026-12-31 | Non-Exclusive | Active |
| LIC018    | The Last Horizon               | MediaHub          | 2026-12-31 | Non-Exclusive | Active |
| LIC019    | Ocean Adventures               | Netflix           | 2026-12-31 | Non-Exclusive | Active |
+-----------+--------------------------------+-------------------+------------+--------------+--------+
9 rows in set (0.00 sec)
```

Figure 7.9: Licensing Analysis Queries and Execution Results example 1

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     L1.LicenseID AS License1,
    ->     L2.LicenseID AS License2,
    ->     A.Title AS AssetTitle,
    ->     T.Name AS Territory,
    ->     L1.StartDate AS License1Start,
    ->     L1.EndDate AS License1End,
    ->     L2.StartDate AS License2Start,
    ->     L2.EndDate AS License2End
    -> FROM LICENSE L1
    -> JOIN LICENSE L2
    ->     ON L1.AssetID = L2.AssetID
    ->     AND L1.LicenseID < L2.LicenseID
    ->     AND L1.Exclusivity = 'Exclusive'
    ->     AND L2.Exclusivity = 'Exclusive'
    ->     AND L1.StartDate <= L2.EndDate
    ->     AND L2.StartDate <= L1.EndDate
    -> JOIN ASSET A
    ->     ON L1.AssetID = A.AssetID
    -> JOIN TERRITORY T
    ->     ON L1.TerritoryID = T.TerritoryID
    -> WHERE L1.TerritoryID = L2.TerritoryID;
Empty set (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
```

Figure 7.10: Licensing Analysis Queries and Execution Results example 2

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

Advanced SQL concepts were implemented in the Digital Rights Management and Asset Licensing Ledger to automate database operations, simplify complex queries, and maintain data consistency.

The implementation includes a stored procedure, stored function, trigger, views, and transaction management.

8.1 Stored Procedure — GetAssetLicenses

A stored procedure is a collection of SQL statements stored in the database that can be executed whenever required.

The GetAssetLicenses stored procedure retrieves licensing agreements associated with a specified digital asset.

It accepts an Asset ID as an input parameter and returns the corresponding licensing information. This eliminates the need to repeatedly write complex SQL queries when retrieving licenses for a particular asset.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER //
(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE PROCEDURE GetAssetLicenses(IN p_AssetID VARCHAR(50))
    -> BEGIN
    ->     SELECT
    ->         L.LicenseID,
    ->         A.Title AS AssetTitle,
    ->         LE.Name AS Licensee,
    ->         T.Name AS Territory,
    ->         L.StartDate,
    ->         L.EndDate,
    ->         L.LicenseType,
    ->         L.Exclusivity,
    ->         L.Status
    ->     FROM LICENSE L
    ->     JOIN ASSET A
    ->         ON L.AssetID = A.AssetID
    ->     JOIN LICENSEE LE
    ->         ON L.LicenseeID = LE.LicenseeID
    ->     JOIN TERRITORY T
    ->         ON L.TerritoryID = T.TerritoryID
    ->     WHERE L.AssetID = p_AssetID
    ->     ORDER BY L.StartDate;
    -> END //
Query OK, 0 rows affected (0.04 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER ;
(PES2UG24CS488@root@localhost) [drm_licensing] > CALL GetAssetLicenses('AST001');
+-----------+--------------+-----------+---------------+-----------+-----------+-------------+-------------+--------+
| LicenseID | AssetTitle   | Licensee  | Territory     | StartDate | EndDate   | LicenseType | Exclusivity | Status |
+-----------+--------------+-----------+---------------+-----------+-----------+-------------+-------------+--------+
| LIC001    | The Last Hor | Netflix   | United States | 2026-01-0 | 2026-12-3 | Streaming   | Exclusive   | Active |
| LIC002    | The Last Hor | Amazon Pr | European Uni | 2026-02-0 | 2026-11-3 | Streaming   | Non-Exclusi | Active |
| LIC018    | The Last Hor | MediaHub  | United Kingd | 2026-06-0 | 2026-12-3 | Distributio | Non-Exclusi | Active |
+-----------+--------------+-----------+---------------+-----------+-----------+-------------+-------------+--------+
3 rows in set (0.03 sec)

Query OK, 0 rows affected (0.07 sec)
```

Figure 8.1: Stored Procedure GetAssetLicenses and Execution Result

8.2 Stored Function — CheckLicenseAvailability

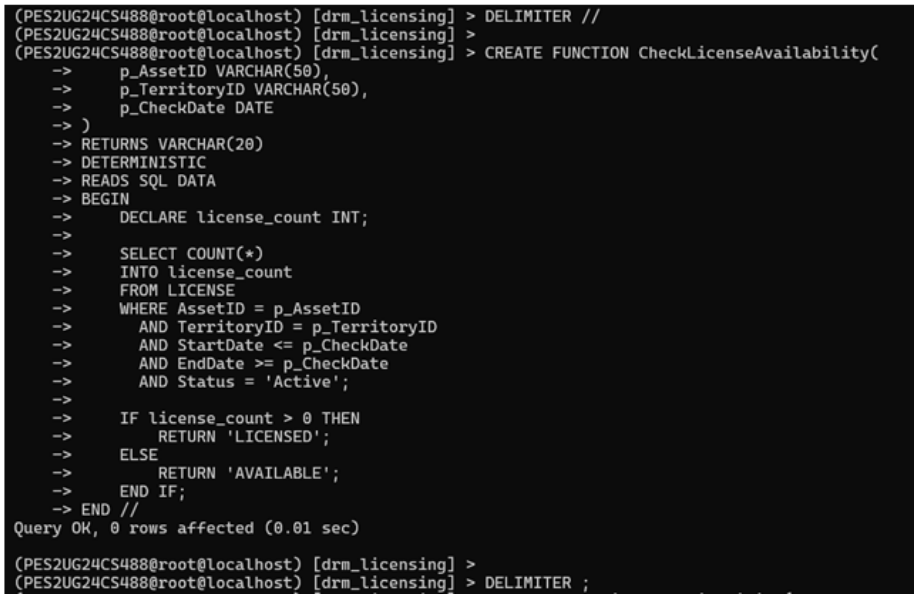
A stored function is a reusable SQL routine that accepts input parameters and returns a value.

The CheckLicenseAvailability function determines whether a digital asset is already licensed in a particular territory on a specified date. It accepts three parameters: Asset ID, Territory ID, and Date.

The function returns:

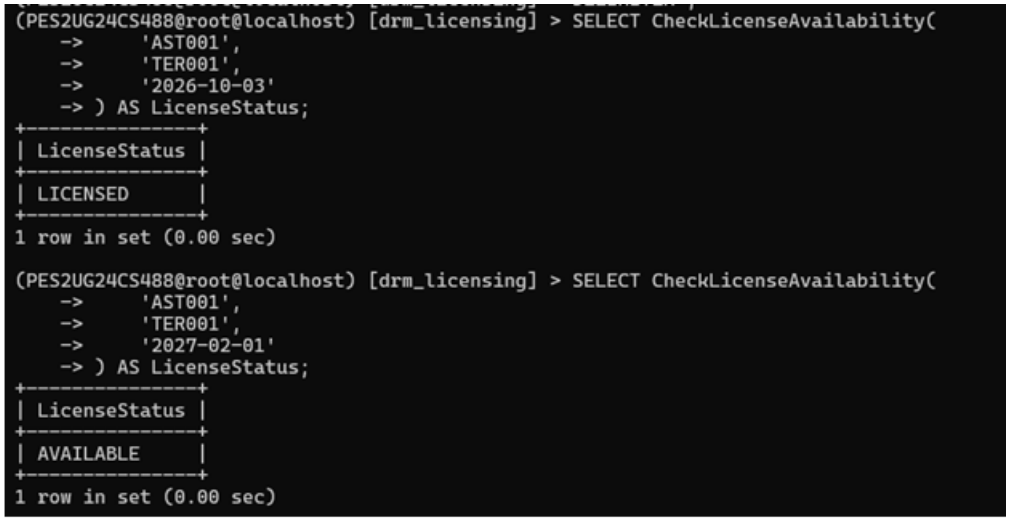
- LICENSED — If an existing license covers the specified asset, territory, and date.
- AVAILABLE — If no matching license is found.

This function helps identify existing licensing agreements before creating new ones.



```
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER //
(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE FUNCTION CheckLicenseAvailability(
    ->     p_AssetID VARCHAR(50),
    ->     p_TerritoryID VARCHAR(50),
    ->     p_CheckDate DATE
    -> )
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE license_count INT;
    ->
    ->     SELECT COUNT(*)
    ->     INTO license_count
    ->     FROM LICENSE
    ->     WHERE AssetID = p_AssetID
    ->       AND TerritoryID = p_TerritoryID
    ->       AND StartDate <= p_CheckDate
    ->       AND EndDate >= p_CheckDate
    ->       AND Status = 'Active';
    ->
    ->     IF license_count > 0 THEN
    ->         RETURN 'LICENSED';
    ->     ELSE
    ->         RETURN 'AVAILABLE';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER ;
```



```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT CheckLicenseAvailability(
    ->     'AST001',
    ->     'TER001',
    ->     '2026-10-03'
    -> ) AS LicenseStatus;
+---------------+
| LicenseStatus |
+---------------+
| LICENSED      |
+---------------+
1 row in set (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT CheckLicenseAvailability(
    ->     'AST001',
    ->     'TER001',
    ->     '2027-02-01'
    -> ) AS LicenseStatus;
+---------------+
| LicenseStatus |
+---------------+
| AVAILABLE     |
+---------------+
1 row in set (0.00 sec)
```

Figure 8.2: Stored Function CheckLicenseAvailability and Execution Result

8.3 Trigger — trg_renewal_history

A trigger is a database object that automatically executes when a specified event occurs on a table.

The trg_renewal_history trigger was implemented to maintain a history of license renewals.

Whenever a new record is inserted into the RENEWAL table, the trigger automatically inserts a corresponding record into LICENSE_HISTORY.

This ensures that renewal activities are recorded without requiring a separate manual insertion.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER //
(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE TRIGGER trg_renewal_history
    -> AFTER INSERT ON RENEWAL
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO HISTORY_SEQUENCE VALUES (NULL);
    ->
    ->     INSERT INTO LICENSE_HISTORY
    ->     (
    ->         HistoryID,
    ->         LicenseID,
    ->         Action,
    ->         ActionDate,
    ->         Remarks
    ->     )
    ->     VALUES
    ->     (
    ->         CONCAT(
    ->             'HIS',
    ->             LPAD(LAST_INSERT_ID(), 3, '0')
    ->         ),
    ->         NEW.LicenseID,
    ->         'Renewal Created',
    ->         NOW(),
    ->         CONCAT(
    ->             'Renewal created from ',
    ->             DATE_FORMAT(NEW.NewStartDate, '%Y-%m-%d'),
    ->             ' to ',
    ->             DATE_FORMAT(NEW.NewEndDate, '%Y-%m-%d')
    ->         )
    ->     );
    -> END //
Query OK, 0 rows affected (0.03 sec)
```

```
(PES2UG24CS488@root@localhost) [drm_licensing] > DELIMITER ;
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO RENEWAL
    -> (
    ->     RenewalID,
    ->     LicenseID,
    ->     RenewalDate,
    ->     NewStartDate,
    ->     NewEndDate,
    ->     Status
    -> )
    -> VALUES
    -> (
    ->     'REN009',
    ->     'LIC013',
    ->     '2026-10-03',
    ->     '2026-11-01',
    ->     '2027-10-31',
    ->     'Approved'
    -> );
Query OK, 1 row affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT *
    -> FROM LICENSE_HISTORY
    -> WHERE LicenseID = 'LIC013'
    -> ORDER BY ActionDate DESC;
+-----------+-----------+-----------------+---------------------+-----------------------------------------+
| HistoryID | LicenseID | Action          | ActionDate          | Remarks                                 |
+-----------+-----------+-----------------+---------------------+-----------------------------------------+
| HIS031    | LIC013    | Renewal Created | 2026-10-03 18:04:48 | Renewal created from 2026-11-01 to 2027-10-31 |
| HIS022    | LIC013    | License Created | 2026-04-01 10:00:00 | European music license created          |
+-----------+-----------+-----------------+---------------------+-----------------------------------------+
2 rows in set (0.00 sec)
```

Figure 8.3: Trigger trg_renewal_history and Execution Result

8.4 SQL Views

SQL views are virtual tables created using SELECT queries. They simplify frequently executed queries and provide a convenient way to retrieve related information.

Two views were implemented in the project.

1. active_license_overview

This view combines licensing information from related tables to display active licensing agreements along with their associated asset, licensee, and territory details.

It simplifies the retrieval of licensing information without repeatedly executing multiple JOIN operations.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE VIEW Active_License_Overview AS
    -> SELECT
    ->     L.LicenseID,
    ->     A.Title AS AssetTitle,
    ->     O.Name AS Owner,
    ->     LE.Name AS Licensee,
    ->     T.Name AS Territory,
    ->     L.StartDate,
    ->     L.EndDate,
    ->     L.LicenseType,
    ->     L.Exclusivity
    -> FROM LICENSE L
    -> JOIN ASSET A
    ->     ON L.AssetID = A.AssetID
    -> JOIN OWNER O
    ->     ON A.OwnerID = O.OwnerID
    -> JOIN LICENSEE LE
    ->     ON L.LicenseeID = LE.LicenseeID
    -> JOIN TERRITORY T
    ->     ON L.TerritoryID = T.TerritoryID
    -> WHERE L.Status = 'Active';
Query OK, 0 rows affected (0.04 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT *
    -> FROM Active_License_Overview
    -> WHERE Exclusivity = 'Exclusive'
    -> ORDER BY EndDate;
+-----------+---------------------------+-----------------+-----------------+----------------+------------+------------+-----------------------+-------------+
| LicenseID | AssetTitle                | Owner           | Licensee        | Territory      | StartDate  | EndDate    | LicenseType           | Exclusivity |
+-----------+---------------------------+-----------------+-----------------+----------------+------------+------------+-----------------------+-------------+
| LIC001    | The Last Horizon          | Warner Media    | Netflix         | United States  | 2026-01-01 | 2026-12-31 | Streaming             | Exclusive   |
| LIC011    | Artificial Intelligence Basics | Penguin Publishing | BookWorld | United Kingdom | 2026-01-01 | 2026-12-31 | Digital Distribution | Exclusive |
| LIC008    | Ocean Adventures          | Pixar Studios   | Amazon Prime Video | Worldwide   | 2026-01-01 | 2027-01-31 | Streaming             | Exclusive   |
| LIC005    | PhotoEditor Pro           | Adobe Systems   | TechSoft Solutions | India       | 2026-01-01 | 2027-02-28 | Software              | Exclusive   |
| LIC015    | Hidden Truth              | Warner Media    | Amazon Prime Video | Australia   | 2026-06-01 | 2027-05-31 | Streaming             | Exclusive   |
+-----------+---------------------------+-----------------+-----------------+----------------+------------+------------+-----------------------+-------------+
5 rows in set (0.00 sec)
```

Figure 8.4: SQL View active_license_overview

2. license_revenue_summary

This view summarizes licensing revenue information using payment records and related licensing details.

It supports financial analysis and helps identify revenue generated through licensing agreements.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE VIEW License_Revenue_Summary AS
    -> SELECT
    ->     L.LicenseID,
    ->     A.Title AS AssetTitle,
    ->     LE.Name AS Licensee,
    ->     COALESCE(SUM(P.Amount), 0) AS TotalPaid
    -> FROM LICENSE L
    -> JOIN ASSET A
    ->     ON L.AssetID = A.AssetID
    -> JOIN LICENSEE LE
    ->     ON L.LicenseeID = LE.LicenseeID
    -> LEFT JOIN PAYMENT P
    ->     ON L.LicenseID = P.LicenseID
    -> GROUP BY
    ->     L.LicenseID,
    ->     A.Title,
    ->     LE.Name;
Query OK, 0 rows affected (0.04 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT *
    -> FROM License_Revenue_Summary
    -> ORDER BY TotalPaid DESC;
+-----------+-----------------------------+--------------------+-----------+
| LicenseID | AssetTitle                  | Licensee           | TotalPaid |
+-----------+-----------------------------+--------------------+-----------+
| LIC008    | Ocean Adventures            | Amazon Prime Video |  90000.00 |
| LIC001    | The Last Horizon            | Netflix            |  60000.00 |
| LIC015    | Hidden Truth                | Amazon Prime Video |  50000.00 |
| LIC002    | The Last Horizon            | Amazon Prime Video |  35000.00 |
| LIC007    | Digital Shadows             | Netflix            |  30000.00 |
| LIC003    | Summer Nights               | Spotify            |  30000.00 |
| LIC017    | SecureCode Toolkit          | TechSoft Solutions |  30000.00 |
| LIC012    | Robot World                 | Netflix            |  28000.00 |
| LIC009    | Midnight Drive              | Spotify            |  22000.00 |
| LIC019    | Ocean Adventures            | Netflix            |  20000.00 |
| LIC005    | PhotoEditor Pro             | TechSoft Solutions |  20000.00 |
| LIC010    | DataViz Studio              | TechSoft Solutions |  18000.00 |
| LIC013    | Echoes of Time              | Spotify            |  16000.00 |
| LIC018    | The Last Horizon            | MediaHub           |  14000.00 |
| LIC011    | Artificial Intelligence Basics | BookWorld       |  12000.00 |
| LIC004    | Summer Nights               | MediaHub           |  12000.00 |
| LIC014    | City Lights                 | MediaHub           |  10000.00 |
| LIC016    | Cloud Computing Guide       | BookWorld          |   9000.00 |
| LIC006    | Database Systems Handbook   | BookWorld          |   8000.00 |
| LIC020    | Hidden Truth                | MediaHub           |      0.00 |
+-----------+-----------------------------+--------------------+-----------+
20 rows in set (0.00 sec)
```

Figure 8.5: SQL View license_revenue_summary

8.5 Transaction Management

Transaction management ensures that multiple related database operations are executed as a single logical unit.

A transaction was implemented for processing license renewals.

The renewal process involves two operations:

1. Inserting a new renewal record into the RENEWAL table.
2. Updating the corresponding license validity period in the LICENSE table.

These operations are executed within a transaction. If both operations succeed, the transaction is committed. If an error occurs, the transaction can be rolled back to prevent inconsistent data.

This demonstrates the use of transaction management to maintain database consistency.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > INSERT INTO RENEWAL
    -> (
    ->     RenewalID,
    ->     LicenseID,
    ->     RenewalDate,
    ->     NewStartDate,
    ->     NewEndDate,
    ->     Status
    -> )
    -> VALUES
    -> (
    ->     'REN010',
    ->     'LIC016',
    ->     '2026-10-03',
    ->     '2027-02-15',
    ->     '2028-02-14',
    ->     'Approved'
    -> );
Query OK, 1 row affected (0.02 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > UPDATE LICENSE
    -> SET
    ->     StartDate = '2027-02-15',
    ->     EndDate = '2028-02-14',
    ->     Status = 'Active'
    -> WHERE LicenseID = 'LIC016';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > COMMIT;
Query OK, 0 rows affected (0.02 sec)
```

```
(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT
    ->     LicenseID,
    ->     StartDate,
    ->     EndDate,
    ->     Status
    -> FROM LICENSE
    -> WHERE LicenseID = 'LIC016';
+-----------+------------+------------+--------+
| LicenseID | StartDate  | EndDate    | Status |
+-----------+------------+------------+--------+
| LIC016    | 2027-02-15 | 2028-02-14 | Active |
+-----------+------------+------------+--------+
1 row in set (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SELECT *
    -> FROM LICENSE_HISTORY
    -> WHERE LicenseID = 'LIC016'
    -> ORDER BY ActionDate DESC;
+-----------+-----------+-----------------+---------------------+-----------------------------------------------+
| HistoryID | LicenseID | Action          | ActionDate          | Remarks                                       |
+-----------+-----------+-----------------+---------------------+-----------------------------------------------+
| HIS032    | LIC016    | Renewal Created | 2026-10-03 18:10:37 | Renewal created from 2027-02-15 to 2028-02-14 |
| HIS026    | LIC016    | License Created | 2026-02-15 11:00:00 | India digital distribution license            |
+-----------+-----------+-----------------+---------------------+-----------------------------------------------+
2 rows in set (0.00 sec)
```

Figure 8.6: SQL Transaction for License Renewal

9. FRONT END DEVELOPMENT

The Digital Rights Management and Asset Licensing Ledger was developed using Next.js to provide an interactive, web-based interface for managing digital assets and licensing operations.

The application allows users to access database information, perform authorized operations, monitor licensing activities, and view the results through a centralized interface.

9.1 Technology Used

The frontend was developed using Next.js and React. The application communicates with backend API routes that interact with MySQL and MongoDB.

MySQL stores structured information such as assets, licenses, payments, and renewals, while MongoDB stores access logs and licensing violation reports.

The interface provides forms, tables, search functionality, and dashboard components to simplify database operations.

9.2 User Authentication and Role-Based Interface

The application implements authentication for two types of users:

1. **System Administrator:** Responsible for managing master data, monitoring licensing information, viewing financial reports, and managing application users.
2. **License Manager:** Responsible for creating and updating licensing agreements, processing renewals and payments, simulating asset access, and resolving licensing violations.

After successful authentication, users are directed to the appropriate dashboard based on their assigned role. Access to application features is restricted according to user permissions.

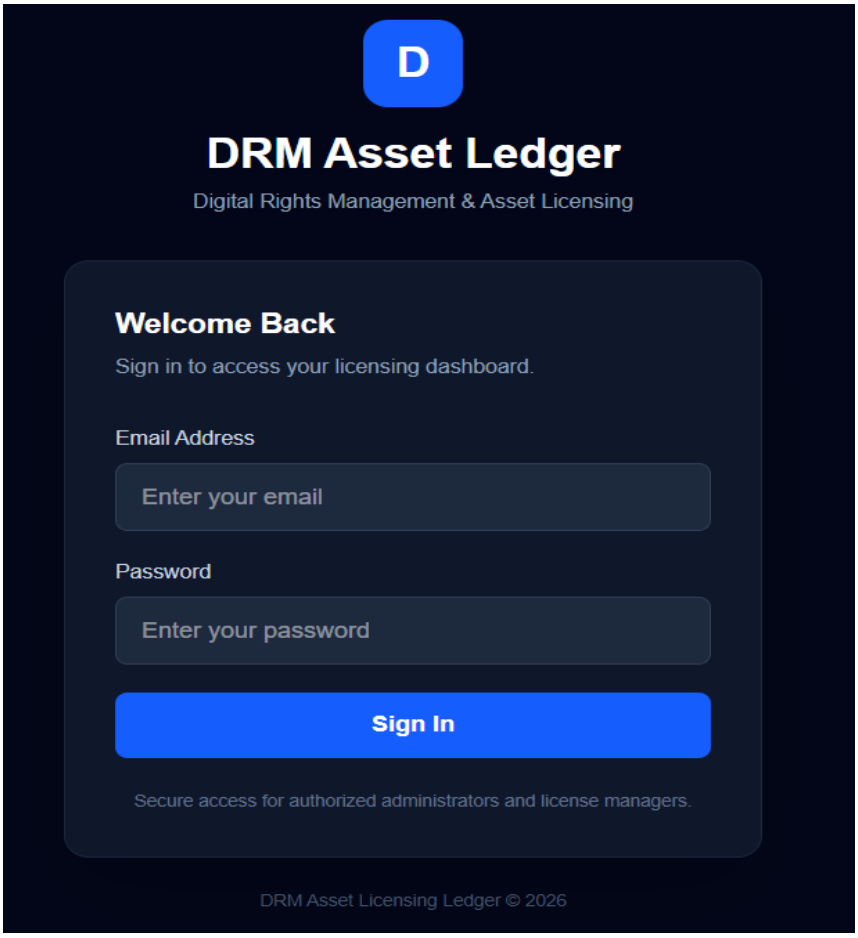


Figure 9.1: Application Login Page

9.3 System Administrator Dashboard

The System Administrator dashboard provides access to the major administrative and monitoring functions of the system.

It includes modules for managing owners, licensees, territories, rights, and digital assets. The administrator can also monitor licensing agreements, renewals, payments, access logs, and violation reports.

Additional dashboard features provide licensing statistics, revenue analysis, and information about expiring or conflicting licenses.

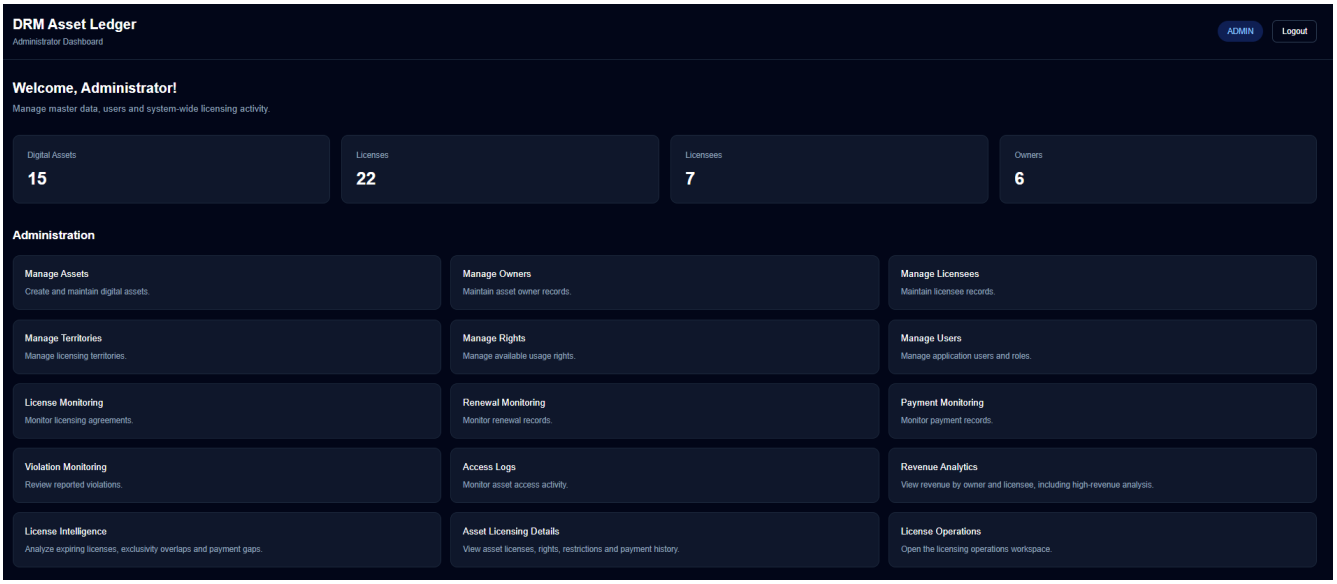


Figure 9.2: System Administrator Dashboard

9.4 Master Data Management

The application provides interfaces to perform Create, Read, Update, and Delete (CRUD) operations on the primary database entities.

Users with the required permissions can add new records, view existing information, modify selected attributes, and delete records where permitted.

Input validation and database constraints help maintain the consistency of the stored information.

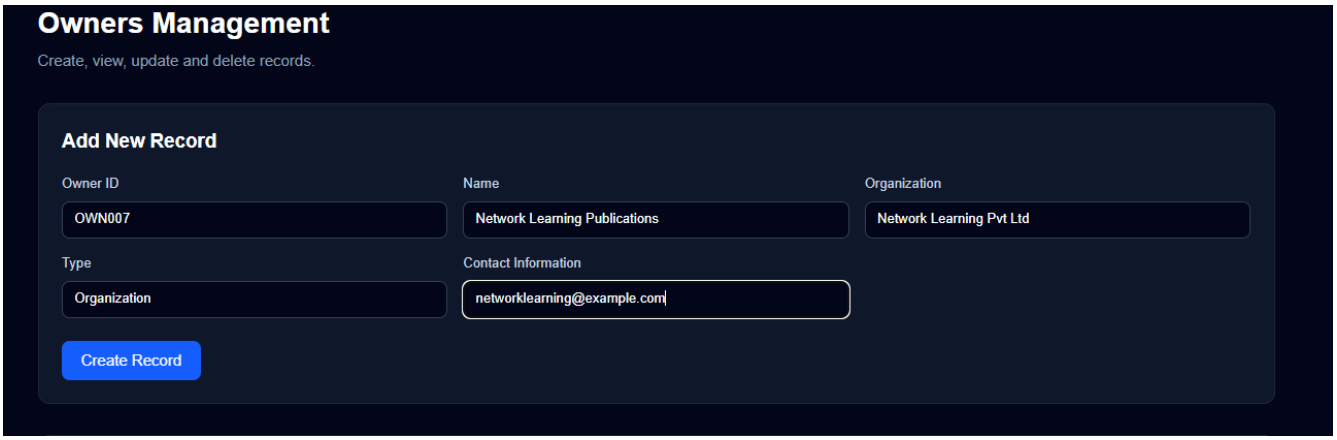


Figure 9.3: Creating a new owner in admin dashboard

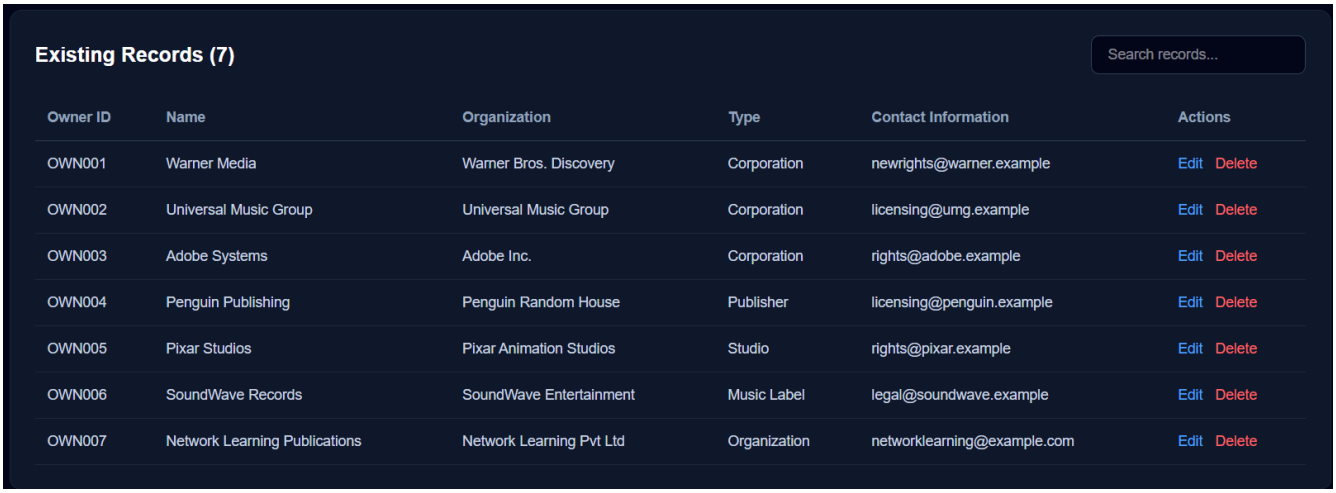


Fig 9.4: Successful creation of OWN007 (owner ID 7)

9.5 License Manager Dashboard

The License Manager dashboard provides an interface for performing day-to-day licensing operations.

The available features include creating licensing agreements, updating existing licenses, processing renewals, recording payments, simulating digital asset access, and resolving reported violations.

These operations communicate with the backend APIs to retrieve or modify the corresponding database records.

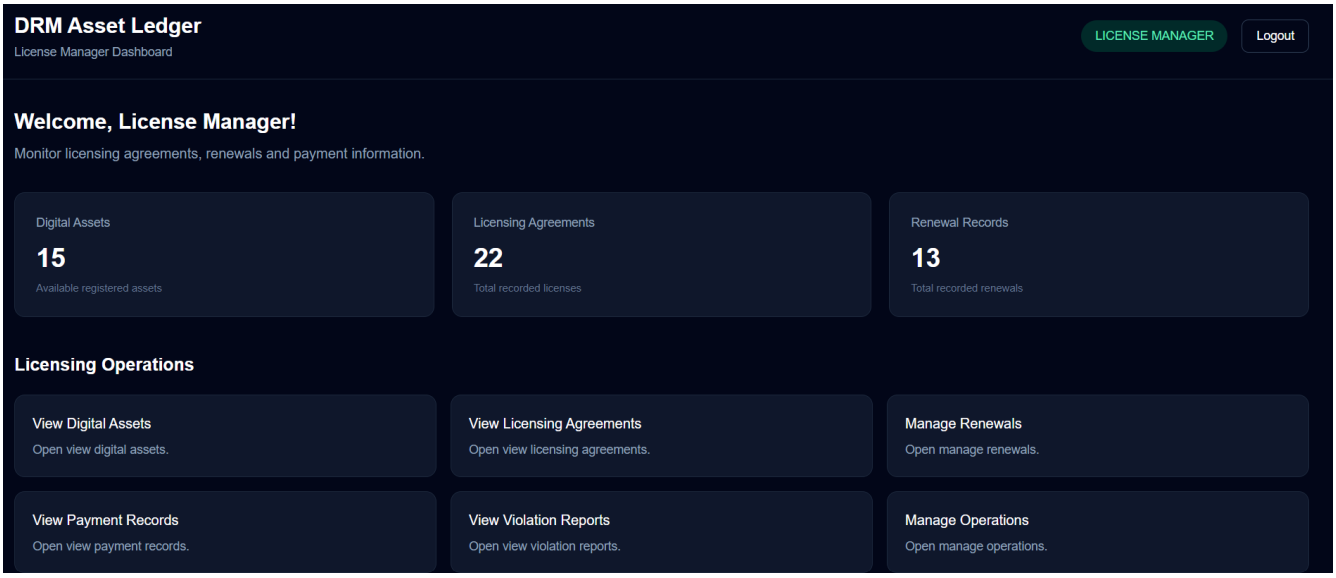


Figure 9.5: License Manager Dashboard

9.6 Licensing and Financial Operations

The application provides forms for creating and updating licensing agreements, including details such as the asset, licensee, territory, validity period, and licensing rights.

Separate interfaces support license renewals and payment recording.

These operations are integrated with the MySQL database to ensure that changes are reflected in the corresponding relational tables.




```
(PES2UG24CS488@root@localhost) [drm_licensing] > select * from license where licenseid='LIC024';
+-----------+---------+------------+-------------+------------+------------+-------------+-------------+--------+--------------+-----------+
| LicenseID | AssetID | LicenseeID | TerritoryID | StartDate  | EndDate    | LicenseType | Exclusivity | Status | Notes        | AutoRenew |
+-----------+---------+------------+-------------+------------+------------+-------------+-------------+--------+--------------+-----------+
| LIC024    | AST001  | LES002     | TER002      | 2027-01-01 | 2027-12-31 | Exclusive   | Exclusive   | Active | Demo license |         0 |
+-----------+---------+------------+-------------+------------+------------+-------------+-------------+--------+--------------+-----------+
1 row in set (0.00 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
```

Figure 9.6: Licensing Agreement Management

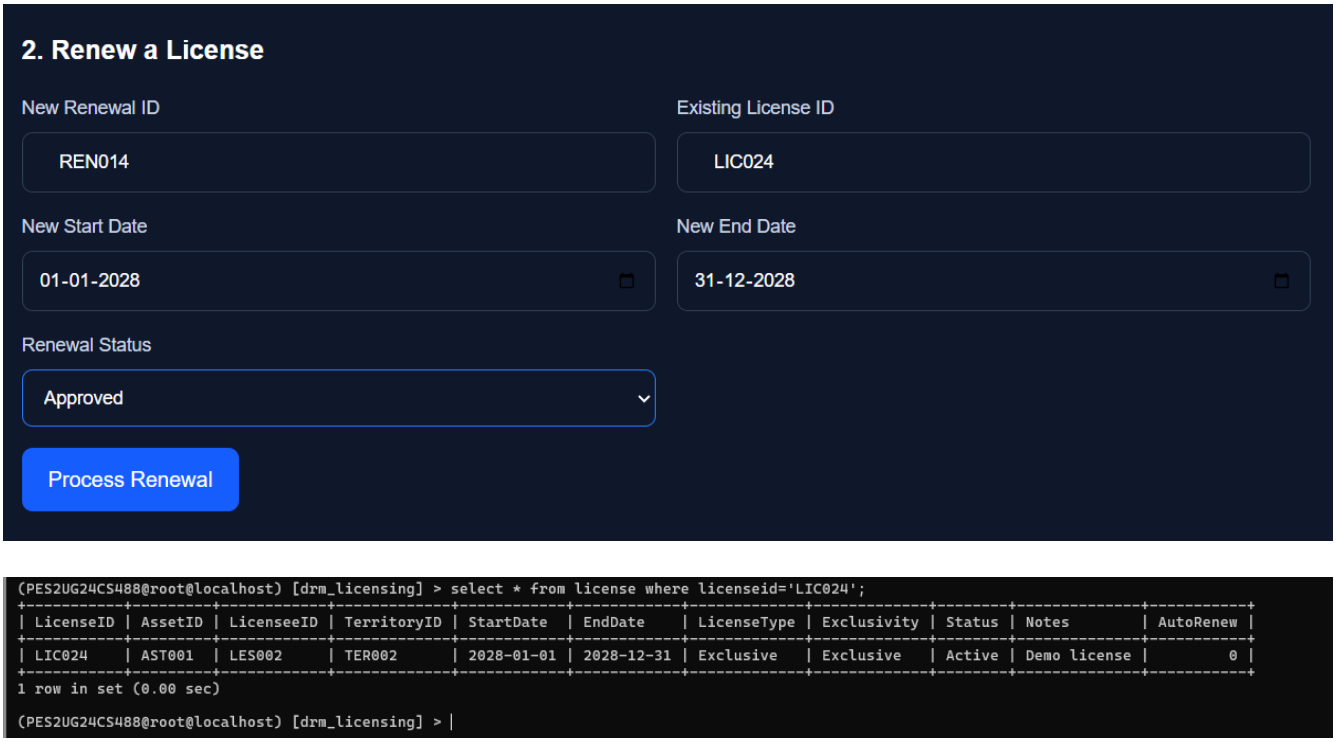


Figure 9.7: Renewal and Payment Operations

9.7 Asset Access Simulation and Violation Monitoring

The application includes an access simulation feature that evaluates whether a requested digital asset access is permitted under the applicable licensing conditions.

The result of each simulated access attempt is recorded in MongoDB as either ALLOWED or DENIED.

When a licensing violation is detected, the system generates a violation report containing the relevant details.

Authorized users can review violation reports and resolve them by updating their status and providing resolution remarks.

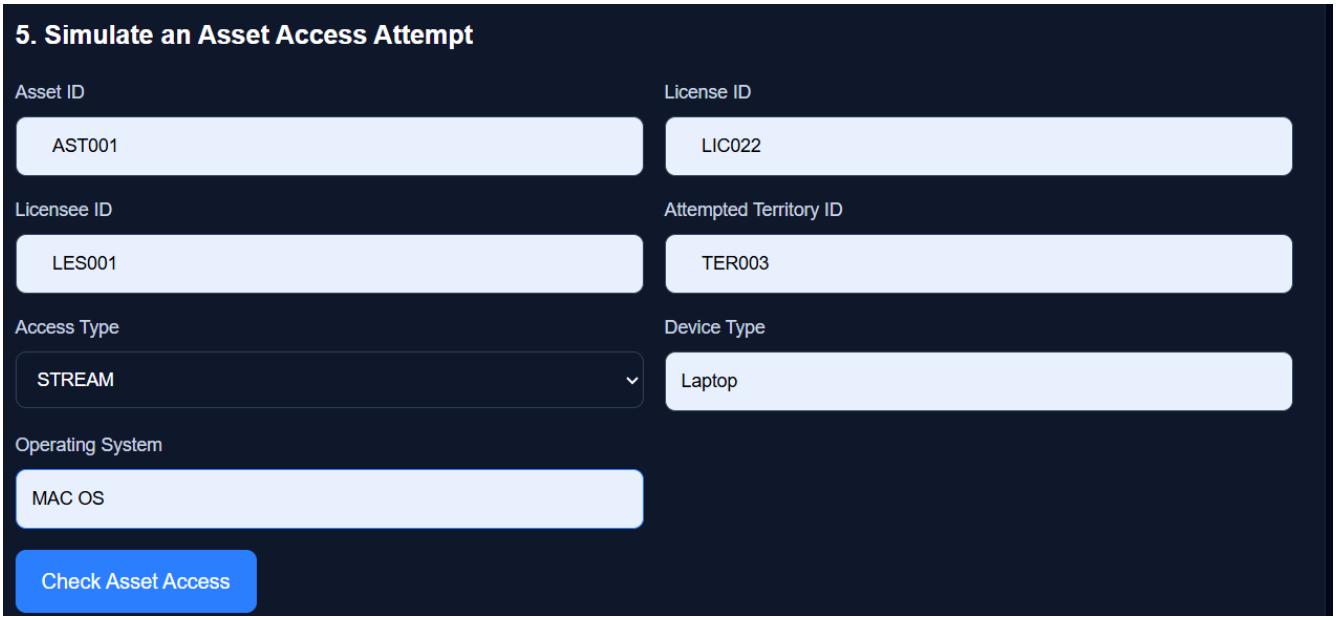


Figure 9.8: Digital Asset Access Simulation



Figure 9.9: Violation Monitoring (1st entry)

9.8 Backend and Database Integration

The Next.js application communicates with backend API routes to perform database operations.

The backend processes user requests, validates input data, applies access restrictions, and executes the required database operations.

MySQL is used for relational data management and advanced SQL operations, while MongoDB handles access events and violation records.

The results are returned to the frontend and displayed through tables, dashboards, and appropriate success or error messages.

This integration provides a unified interface for interacting with both relational and NoSQL databases.

10. MONGODB IMPLEMENTATION

MongoDB was integrated into the Digital Rights Management and Asset Licensing Ledger to store access activity and licensing violation reports.

Unlike MySQL, which stores structured licensing information in relational tables, MongoDB uses a document-oriented data model. This makes it suitable for recording access events and violation details that may contain varying amounts of information.

The MongoDB database was created with the name drm_licensing and contains two main collections: access_logs and violation_reports.

10.1 Technology Justification

MySQL was selected for managing structured licensing information because it supports relationships, foreign key constraints, transactions, and advanced SQL operations.

MongoDB was selected for storing access logs and violation reports because its flexible document structure allows event details, access decisions, and violation evidence to be maintained efficiently.

Using both databases allows the application to manage structured licensing records and event-based monitoring information separately.

10.2 MongoDB Database and Collections

The MongoDB database consists of the following collections:

1. access_logs

This collection stores records of simulated digital asset access attempts.

Each document contains information such as the access event identifier, associated asset and license, licensee, attempted territory, access type, timestamp, and access decision.

The access decision indicates whether the request was ALLOWED or DENIED.

2. violation_reports

This collection stores reports generated when an access attempt violates the applicable licensing conditions.

Each report contains information such as the violation identifier, associated asset and license, violation type, severity, detection timestamp, evidence, and resolution status.

The reports are initially recorded with an OPEN status and can subsequently be marked as RESOLVED.

```
C:\Users\shreyas-pc>mongosh "mongodb://localhost:27017" -u drm_app -p --authenticationDatabase drm_licensing
Enter password: ***********
Current Mongosh Log ID: 6aecca8554c2cf700f63e7e4
Connecting to:          mongodb://<credentials>@localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000&aut
hSource=drm_licensing&appName=mongosh+2.14.1
Using MongoDB:          7.0.43
Using Mongosh:          2.14.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> use drm_licensing
switched to db drm_licensing
drm_licensing> show collections
access_logs
violation_reports
drm_licensing>
```

Figure 10.1: MongoDB Database and Collections

10.3 Access Log Operations

The application records simulated asset access attempts in the access_logs collection.

When an access request is submitted, the backend validates the relevant licensing information against the MySQL database.

Based on the validation result an access log document is inserted into MongoDB with the appropriate decision.The stored documents can be retrieved to monitor access activity and identify denied requests.

```
drm_licensing> db.access_logs.find().pretty()
[
  {
    _id: ObjectId('6ae83891d737efce72b3d978'),
    eventId: 'EVT001',
    assetId: 'AST001',
    licenseId: 'LIC001',
    licenseeId: 'LES001',
    eventType: 'STREAM',
    eventDate: ISODate('2026-11-02T08:00:00.000Z'),
    territoryId: 'TER001',
    result: 'ALLOWED',
    denialReason: null,
    device: { deviceType: 'Laptop', platform: 'Windows' },
    metadata: { ipAddress: '192.168.1.10', accessChannel: 'Web' }
  },
  {
    _id: ObjectId('6ae83891d737efce72b3d979'),
    eventId: 'EVT002',
    assetId: 'AST001',
    licenseId: 'LIC001',
    licenseeId: 'LES001',
    eventType: 'DOWNLOAD',
    eventDate: ISODate('2026-11-02T08:15:00.000Z'),
    territoryId: 'TER003',
    result: 'DENIED',
    denialReason: 'TERRITORY_MISMATCH',
    device: { deviceType: 'Mobile', platform: 'Android' },
    metadata: { ipAddress: '192.168.1.20', accessChannel: 'Mobile App' }
  },
```

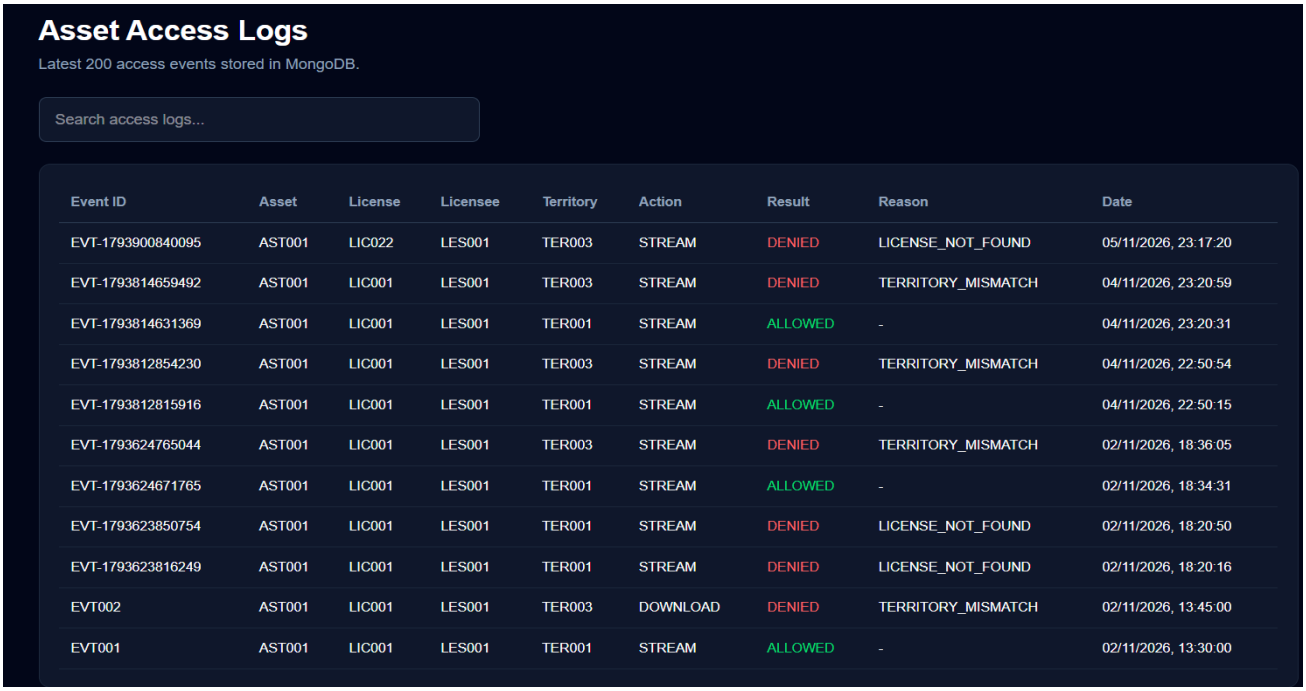


Figure 10.2: MongoDB Access Log Documents

10.4 Violation Report Generation

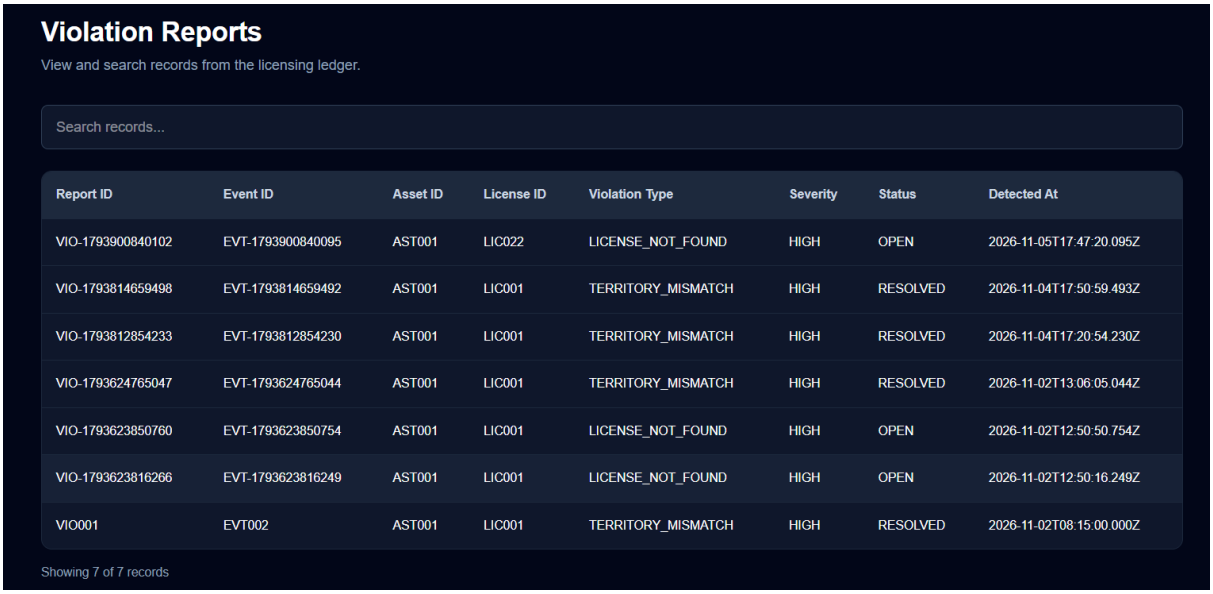
When a simulated access attempt violates licensing conditions, the application generates a violation report.

For example, a territory mismatch occurs when an asset is accessed from a territory different from the one permitted by the licensing agreement.

The corresponding document is stored in the violation_reports collection with the violation type, severity, evidence, and status.

These reports help administrators identify and monitor potential licensing violations.

```
drm_licensing> db.violation_reports.find().pretty()
[
  {
    _id: ObjectId('6ae83d0ad737efce72b3d97a'),
    reportId: 'VIO001',
    eventId: 'EVT002',
    assetId: 'AST001',
    licenseId: 'LIC001',
    licenseeId: 'LES001',
    violationType: 'TERRITORY_MISMATCH',
    severity: 'HIGH',
    detectedAt: ISODate('2026-11-02T08:15:00.000Z'),
    expectedTerritory: 'TER001',
    attemptedTerritory: 'TER003',
    status: 'RESOLVED',
    evidence: [
      'Access attempt was denied',
      'Requested territory did not match the license territory'
    ],
    resolution: {
      resolvedAt: ISODate('2026-11-02T07:43:02.698Z'),
      remarks: 'Access was denied successfully. Territory mismatch was reviewed and documented.'
    }
  }
```



Violation Reports

View and search records from the licensing ledger.

Search records...

Report ID	Event ID	Asset ID	License ID	Violation Type	Severity	Status	Detected At
VIO-1793900840102	EVT-1793900840095	AST001	LIC022	LICENSE_NOT_FOUND	HIGH	OPEN	2026-11-05T17:47:20.095Z
VIO-1793814659498	EVT-1793814659492	AST001	LIC001	TERRITORY_MISMATCH	HIGH	RESOLVED	2026-11-04T17:50:59.493Z
VIO-1793812854233	EVT-1793812854230	AST001	LIC001	TERRITORY_MISMATCH	HIGH	RESOLVED	2026-11-04T17:20:54.230Z
VIO-1793624765047	EVT-1793624765044	AST001	LIC001	TERRITORY_MISMATCH	HIGH	RESOLVED	2026-11-02T13:06:05.044Z
VIO-1793623850760	EVT-1793623850754	AST001	LIC001	LICENSE_NOT_FOUND	HIGH	OPEN	2026-11-02T12:50:50.754Z
VIO-1793623816266	EVT-1793623816249	AST001	LIC001	LICENSE_NOT_FOUND	HIGH	OPEN	2026-11-02T12:50:16.249Z
VIO001	EVT002	AST001	LIC001	TERRITORY_MISMATCH	HIGH	RESOLVED	2026-11-02T08:15:00.000Z

Showing 7 of 7 records

Figure 10.3: MongoDB Violation Reports

10.5 MongoDB Read, Update and Delete Operations

MongoDB supports document-level operations for retrieving, modifying, and deleting records.

Read Operation: The application retrieves access logs and violation reports from their respective collections to display them in the monitoring interfaces.

Update Operation: Authorized users can update a violation report by changing its status and recording resolution information.

Delete Operation: MongoDB supports removal of documents using operations such as deleteOne() and deleteMany(). Deletion should be demonstrated using a temporary test document to avoid removing actual monitoring records.

These operations demonstrate how MongoDB manages event-based information without requiring a fixed relational schema.

```
drm_licensing> db.violation_reports.countDocuments()
7
drm_licensing>
```

Figure 10.4: MongoDB Read Operation

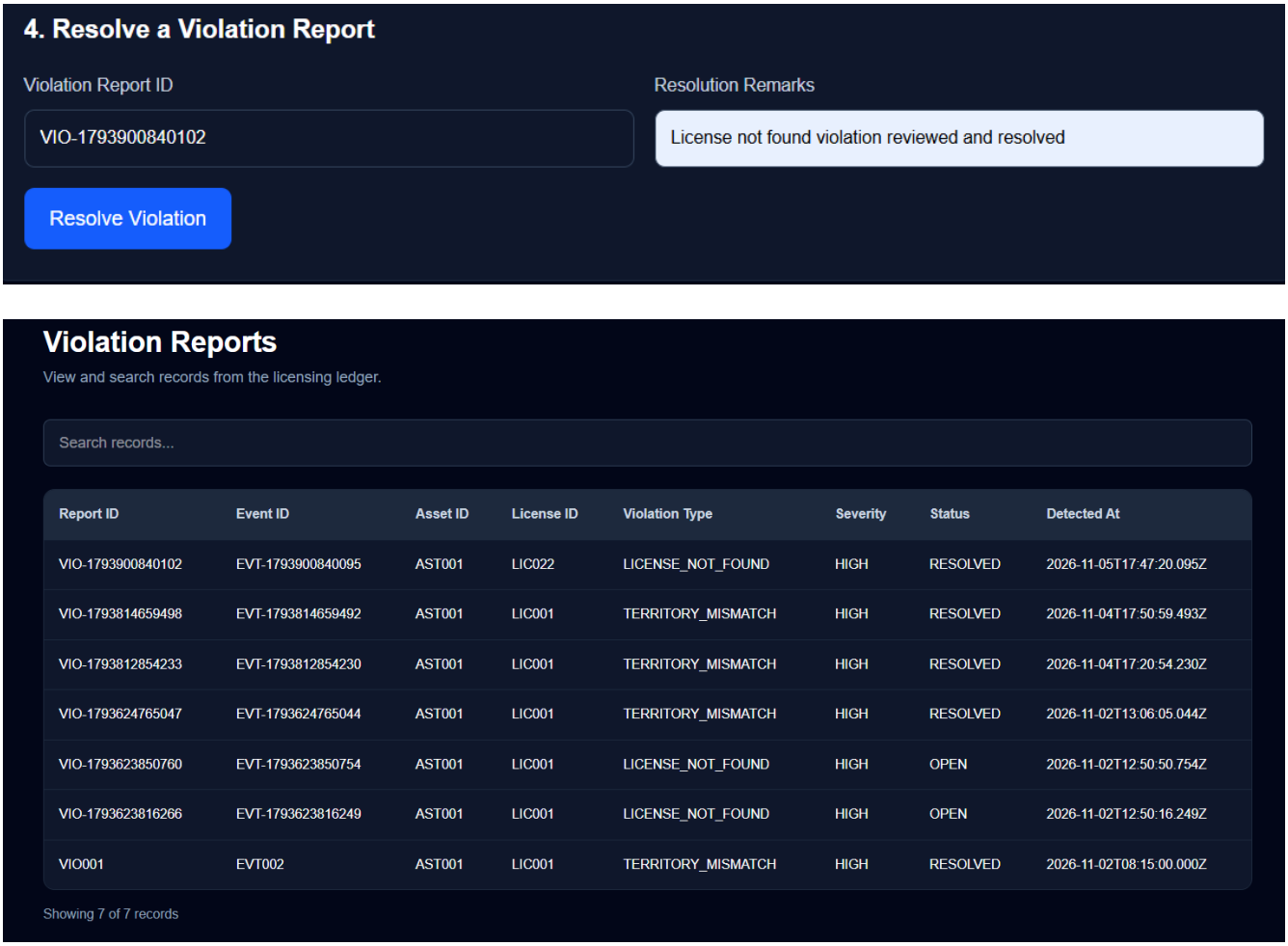


Figure 10.5: MongoDB Update Operation

Please compare the 1st entry of fig 10.3 and 10.5 to verify the update.

```
drm_licensing> db.violation_reports.deleteOne({
|   reportId: "VIO-1793900840102",
|   status: "RESOLVED"
| })
{ acknowledged: true, deletedCount: 1 }
drm_licensing> db.violation_reports.findOne({
|   reportId: "VIO-1793900840102"
| })
null
drm_licensing>
```

Figure 10.6: MongoDB Delete Operation

10.6 Violation Resolution Demonstration

The violation resolution feature was tested using a territory mismatch scenario.

A violation report was generated for asset AST001 and license LIC001, where the permitted territory was TER001, but access was attempted from territory TER003.

The system identified the violation as TERRITORY_MISMATCH with HIGH severity and initially recorded its status as OPEN.

The License Manager subsequently resolved the violation through the application interface by providing resolution remarks.

MongoDB updated the document status to RESOLVED and recorded the resolution timestamp while preserving the original violation evidence.

This demonstrates the integration between the Next.js interface, backend API, and MongoDB document update operations.

```
drm_licensing> db.violation_reports.findOne({
|   reportId: "VIO-1793900840102"
| })
{
  _id: ObjectId('6aecc128f7f65b5d7ca7a013'),
  reportId: 'VIO-1793900840102',
  eventId: 'EVT-1793900840095',
  assetId: 'AST001',
  licenseId: 'LIC022',
  licenseeId: 'LES001',
  violationType: 'LICENSE_NOT_FOUND',
  severity: 'HIGH',
  detectedAt: ISODate('2026-11-05T17:47:20.095Z'),
  expectedTerritory: null,
  attemptedTerritory: 'TER003',
  status: 'RESOLVED',
  evidence: [ 'Simulated access attempt was denied.' ],
  resolution: {
    resolvedAt: ISODate('2026-11-05T18:48:48.058Z'),
    remarks: 'License not found violation reviewed and resolved.'
  }
}
drm_licensing>
```

Figure 10.7: Violation Resolution Before and After Update

10.7 Summary

The MongoDB implementation provides a separate storage mechanism for access monitoring and violation management.

The access_logs collection maintains a history of simulated access attempts, while the violation_reports collection supports the identification, tracking, and resolution of licensing violations.

Together with MySQL, MongoDB enables the application to manage both structured licensing information and event-based monitoring records.

11. AUTHENTICATION AND ROLE-BASED ACCESS CONTROL

The Digital Rights Management and Asset Licensing Ledger implements authentication and Role-Based Access Control (RBAC) to ensure that users can access only the operations permitted by their assigned roles.

11.1 User Authentication

The application uses the app_user table in MySQL to maintain user account information. Users authenticate through the Next.js login interface, and their credentials are verified by the backend before access is granted.

After successful authentication, users are directed to the dashboard corresponding to their assigned role.

11.2 Application-Level Role-Based Access Control

Two application roles were implemented:

1. **System Administrator:** Manages owners, licensees, territories, rights, assets, and application users. The administrator can also monitor licenses, payments, renewals, access logs, and violation reports.
2. **License Manager:** Manages licensing agreements, processes renewals and payments, simulates asset access, and resolves licensing violations.

Role-based restrictions are enforced through the application interface and backend APIs to prevent unauthorized operations.

11.3 Database-Level Role-Based Access Control

MySQL roles were also created to control database permissions independently of the application interface.

The drm_admin_role and drm_license_manager_role roles were created and assigned to the corresponding database users, drm_admin_user and drm_license_manager_user.

The administrator role provides broader database privileges, while the license manager role is restricted to the permissions required for licensing operations.

This demonstrates access control at both the application and database levels.

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE ROLE 'drm_admin_role', 'drm_license_manager_role'
Query OK, 0 rows affected (0.04 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT ALL PRIVILEGES
    -> ON drm_licensing.*
    -> TO 'drm_admin_role';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT SELECT
    -> ON drm_licensing.*
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.02 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT INSERT, UPDATE
    -> ON drm_licensing.LICENSE
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT INSERT, UPDATE
    -> ON drm_licensing.RENEWAL
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT INSERT, UPDATE
    -> ON drm_licensing.PAYMENT
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT EXECUTE
    -> ON PROCEDURE drm_licensing.GetAssetLicenses
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.02 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT EXECUTE
    -> ON FUNCTION drm_licensing.CheckLicenseAvailability
    -> TO 'drm_license_manager_role';
Query OK, 0 rows affected (0.01 sec)
```

Figure 11.1: MySQL role creation and privileges granting

```
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE USER 'drm_admin_user'@'localhost'
    -> IDENTIFIED BY 'Admin@12345';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > CREATE USER 'drm_license_manager_user'@'localhost'
    -> IDENTIFIED BY 'License@12345';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT 'drm_admin_role'
    -> TO 'drm_admin_user'@'localhost';
Query OK, 0 rows affected (0.02 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > GRANT 'drm_license_manager_role'
    -> TO 'drm_license_manager_user'@'localhost';
Query OK, 0 rows affected (0.03 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] > SET DEFAULT ROLE 'drm_admin_role'
    -> TO 'drm_admin_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

(PES2UG24CS488@root@localhost) [drm_licensing] >
(PES2UG24CS488@root@localhost) [drm_licensing] > SET DEFAULT ROLE 'drm_license_manager_role'
    -> TO 'drm_license_manager_user'@'localhost';
Query OK, 0 rows affected (0.03 sec)
```

Figure 11.2: MySQL user creation

11.4 Summary

The combined implementation of user authentication, application-level RBAC, and MySQL database roles provides controlled access to the system's features and database operations.

12. CONCLUSION

The Digital Rights Management and Asset Licensing Ledger was successfully developed as a database-driven application for managing digital assets, licensing agreements, payments, renewals, and licensing violations.

The project integrates MySQL and MongoDB to handle different types of data. MySQL maintains structured licensing information through relational tables, primary and foreign keys, integrity constraints, and advanced SQL features such as stored procedures, functions, triggers, views, and transactions. MongoDB stores simulated asset access logs and violation reports, supporting document retrieval, updates, and deletion operations.

A web-based interface developed using Next.js enables System Administrators and License Managers to perform their authorized operations. The application integrates frontend functionality with backend APIs and both databases, allowing users to manage licensing records, simulate asset access, monitor violations, and resolve reported incidents

Overall, the project provides a centralized and organized approach to digital rights and licensing management while demonstrating the practical application of relational and NoSQL database technologies.

REFERENCES

1. MySQL Reference Manual — https://dev.mysql.com/doc/

2. MongoDB Documentation — https://www.mongodb.com/docs/

3. Next.js Documentation — https://nextjs.org/docs

GitHub Link: https://github.com/shreyaspatil4106-glitch/Dbms_digital_rights_and_asset_licensing_ledger.git